PRODUCT REQUIREMENTS DOCUMENT

لغة العرب

Lughat Al-Arab

El primer lenguaje de programación nativo en árabe

Versión 1.0 · Marzo 2026 · Proyecto de 3 meses

Autor: A. Khamis

Editor: VSCode (Language Extension + Monaco Core)

Estado: Planificación activa

Índice de Contenidos

1 Resumen Ejecutivo4
2 Contexto y Motivación del Proyecto5
3 Visión del Producto7
4 Usuarios Objetivo y Casos de Uso9
5 Especificación Completa del Lenguaje12
5.1 Filosofía de diseño12
5.2 Sistema de tipos14
5.3 Palabras clave (Keywords)16
5.4 Sintaxis formal — Gramática EBNF19
5.5 Operadores y precedencia24
5.6 Sistema de módulos26
5.7 Manejo de errores27
5.8 Ejemplos de programas completos29
6 Arquitectura del Compilador34
6.1 Pipeline completo34
6.2 Lexer (Análisis léxico)36
6.3 Parser (Análisis sintáctico)40
6.4 AST — Árbol de Sintaxis Abstracta44
6.5 Analizador semántico48
6.6 Intérprete de árbol (MVP)51
6.7 Gestión de memoria53
6.8 Mensajes de error en árabe55
7 IDE — Extensión VSCode57
7.1 Arquitectura de la extensión57
7.2 Soporte RTL en Monaco Editor59
7.3 Language Server Protocol (LSP)61
7.4 Syntax highlighting con TextMate63
8 Stack Tecnológico y Decisiones de Diseño65
9 Roadmap de 12 Semanas68
10 Hitos y Métricas de Éxito72
11 Riesgos y Mitigaciones75
12 Visión a Largo Plazo78
13 Próximos Pasos Inmediatos82
14 Apéndice — Referencia Completa del Lenguaje84

1. Resumen Ejecutivo

Lughat Al-Arab (لغة العرب) es un lenguaje de programación de propósito general diseñado íntegramente en árabe estándar moderno. Es el primer lenguaje en el que los keywords, los mensajes de error, la documentación y el entorno de desarrollo están en árabe, sin transliteraciones ni concesiones al inglés.

El proyecto tiene un horizonte de 3 meses para entregar un MVP funcional: un intérprete capaz de ejecutar programas reales, junto con una extensión de Visual Studio Code que proporcione una experiencia de desarrollo completa en escritura RTL (de derecha a izquierda).

Datos clave del proyecto

Nombre del producto:	لغة العرب (Lughat Al-Arab)
Extensión de archivo:	.عرب
Paradigma:	Multiparadigma (imperativo, OOP, funcional)
Numeración:	Dígitos occidentales 0-9 (claridad máxima)
Árabe:	Estándar moderno simplificado (sin hamza ni diacríticos)
IDE:	Extensión VSCode + Monaco Editor (núcleo de VSCode)
Compilador MVP:	Python 3 (intérprete de árbol)
Compilador v1:	Rust + LLVM (compilación nativa)
Duración del proyecto:	12 semanas (3 meses)
Equipo:	1 desarrollador (A. Khamis)

El MVP se divide en tres fases: (1) Fundamentos del lenguaje y compilador — semanas 1 a 4; (2) Intérprete completo con librería estándar — semanas 5 a 8; (3) IDE en VSCode y pulido — semanas 9 a 12. Al finalizar las 12 semanas el proyecto estará listo para publicación pública en GitHub con una demo funcional.

Criterio de éxito del MVP

Un experto en compiladores debe poder leer este documento y entender exactamente cómo implementar Lughat Al-Arab desde cero, sin necesidad de consultar fuentes externas.

Un programador araboparlante sin experiencia en C o Python debe poder escribir un programa funcional en Lughat Al-Arab en menos de 30 minutos usando solo la documentación.

2. Contexto y Motivación del Proyecto

2.1 El problema lingüístico en la programación

La programación moderna está fundamentalmente anclada al inglés. Los 26 lenguajes de programación más utilizados (según el índice TIOBE 2025) tienen sus palabras clave, mensajes de error y documentación primaria en inglés. Esto crea una barrera cognitiva adicional para los 400 millones de hablantes nativos de árabe que quieren aprender a programar.

El impacto no es trivial. Cuando un estudiante araboparlante encuentra un error como "TypeError: unsupported operand type(s) for +: int and str", debe primero traducir el mensaje mentalmente antes de poder razonar sobre el problema técnico. Este proceso de doble traducción (inglés → árabe → solución) ralentiza el aprendizaje y aumenta la frustración.

El problema real que estamos resolviendo

No es solo que los keywords sean en inglés. Es que todo el ecosistema de programación (errores, documentación, comunidad, tutoriales) asume inglés como idioma nativo.

Un programador araboparlante principiante debe aprender dos cosas a la vez:

1. Los conceptos de programación (variables, funciones, bucles...)
2. El vocabulario técnico en inglés que corresponde a esos conceptos

Lughat Al-Arab elimina la segunda barrera completamente.

2.2 Estado del arte: lenguajes de programación en árabe

Han existido intentos previos de crear lenguajes de programación en árabe, pero ninguno ha alcanzado adopción real:

- ArabiScript (2003): lenguaje de scripting web, abandonado. No tenía tipado, gestión de memoria ni soporte para IA.
- Qalb (2012): lenguaje funcional de Khaled Al-Jabri. Implementación en Haskell, sintaxis minimalista pero sin IDE ni ecosistema. GitHub: ~800 estrellas pero sin actividad desde 2014.
- Al-Kindi (2019): proyecto universitario jordano. Compilaba a JavaScript pero la sintaxis era incómoda y el proyecto quedó sin mantenimiento.
- ArabicPy (2021): wrapper de Python con keywords traducidas. Depende completamente de Python y no es un lenguaje independiente.

Ninguno de estos proyectos combinó: (1) sintaxis árabe nativa legible, (2) compilador propio, (3) IDE profesional, (4) mensajes de error en árabe, y (5) soporte para IA y datos grandes. Lughat Al-Arab es el primer intento de construir los cinco pilares juntos.

2.3 Oportunidad de mercado

El mundo árabe tiene una de las poblaciones jóvenes más grandes del mundo — más del 60% de la población de los países árabes tiene menos de 30 años. La demanda de educación tecnológica está creciendo exponencialmente, y los gobiernos de Arabia Saudí, Emiratos, Egipto y Marruecos han lanzado iniciativas masivas de alfabetización digital.

Lughat Al-Arab no pretende reemplazar Python o Rust. Su objetivo inicial es ser el lenguaje con el que los estudiantes árabes aprendan a programar por primera vez, de la misma manera que Scratch es el lenguaje con el que aprenden los niños en occidente.

2.4 Por qué ahora

- Los LLMs (GPT-4, Claude) han demostrado que el árabe puede ser procesado con alta fidelidad por sistemas de IA — esto valida la hipótesis técnica del proyecto.
- VSCode tiene una API de extensiones madura que permite construir soporte RTL completo sin modificar el núcleo del editor.
- Rust y LLVM han madurado hasta el punto en que un solo desarrollador puede construir un compilador de producción en meses, no años.
- El momento político y cultural es favorable: hay financiación árabe disponible para proyectos de tecnología educativa.

3. Visión del Producto

3.1 Declaración de visión

Lughat Al-Arab es un lenguaje de programación multiparadigma, compilado, con tipado estático opcional, diseñado para ser la primera elección de programadores araboparlantes. El lenguaje está optimizado para tres dominios: educación (claridad y mensajes de error comprensibles), inteligencia artificial (sintaxis nativa para modelos y tensores) y procesamiento de datos a gran escala.

3.2 Los cinco pilares del producto

Pilar 1 — Lenguaje nativo en árabe

Todos los keywords son palabras árabes reales y reconocibles sin necesidad de diccionario. La escritura es de derecha a izquierda. Los números son dígitos occidentales (0-9) porque ya son de uso universal en el mundo árabe moderno y evitan ambigüedades. No se usan hamza (ء) ni diacríticos para maximizar la comodidad de escritura en teclados estándar.

Pilar 2 — Compilador propio de alto rendimiento

El MVP usa un intérprete de árbol escrito en Python para validar el diseño del lenguaje. La versión 1.0 (post-MVP) compilará a binario nativo a través de LLVM, dando acceso a optimizaciones de nivel C. La gestión de memoria seguirá un modelo de ownership simplificado (inspirado en Rust pero con menos rigidez) para evitar garbage collection y garantizar rendimiento predecible.

Pilar 3 — IDE profesional con VSCode

En lugar de construir un IDE desde cero, Lughat Al-Arab aprovecha el núcleo de VSCode (Monaco Editor) a través de una extensión. Esto proporciona inmediatamente: syntax highlighting, autocompletado, navegación de errores, terminal integrada y debugging — todo sin reimplementar miles de líneas de código de UI. La extensión añade soporte RTL nativo y un Language Server Protocol (LSP) propio del lenguaje.

Pilar 4 — Soporte nativo para IA y datos

El lenguaje incluye desde el diseño tipos de datos para tensores y operaciones matriciales. Los keywords درب (entrenar) و تنبا (predecir) son ciudadanos de primera clase, no solo llamadas a librerías. En Fase 3 (post-MVP) se añadirán bindings nativos a kernels BLAS/LAPACK y soporte para pipelines de datos con formato Apache Arrow.

Pilar 5 — Mensajes de error comprensibles en árabe

Los errores del compilador/intérprete están en árabe, son específicos y orientados a la solución. En lugar de "TypeError: unsupported operand type(s) for +", el usuario ve: "خطا في النوع: لا يمكن جمع عدد مع نص في السطر 7 — هل قصدت تحويل النص الى عدد؟". Cada mensaje incluye: tipo de error, ubicación exacta, descripción en árabe, y una sugerencia de solución.

3.3 Qué NO es Lughat Al-Arab

- No es un wrapper de Python con keywords traducidas. Es un compilador independiente con su propio pipeline léxico-sintáctico-semántico.
- No es un lenguaje de scripting web. No compila a JavaScript (al menos en el MVP).
- No requiere conocer árabe clásico o fusha académica. Usa árabe estándar moderno simplificado.
- No reemplaza a Python, Rust o C++. Coexiste con ellos a través de un sistema de FFI (Foreign Function Interface) en versiones futuras.

4. Usuarios Objetivo y Casos de Uso

4.1 Perfiles de usuario

Perfil A — Estudiante araboparlante (usuario primario)

Edad: 14-22 años. Habla árabe como idioma nativo. Tiene conocimiento básico o nulo de inglés técnico. Quiere aprender programación pero las barreras del inglés le ralentizan. Usa un teclado árabe-inglés estándar.

Necesidades: keywords que pueda leer y recordar sin traductor, mensajes de error que entienda inmediatamente, tutoriales en árabe que usen el mismo lenguaje que el código.

Perfil B — Desarrollador profesional araboparlante

Edad: 22-40 años. Programa en Python, JavaScript o Java. Habla inglés técnico con fluidez. Quiere contribuir al ecosistema tecnológico árabe o enseñar programación en árabe.

Necesidades: un lenguaje de alto rendimiento que no sacrifique expresividad, con herramientas de desarrollo profesionales.

Perfil C — Investigador de IA en el mundo árabe

Trabaja en NLP árabe, visión por computador o aprendizaje automático. Necesita un lenguaje que sea expresivo para definir modelos y eficiente para entrenarlos, con documentación en árabe.

Perfil D — Institución educativa

Universidad o instituto que quiere introducir programación en su currículo en árabe. Necesita un lenguaje estable, con documentación completa y soporte a largo plazo.

4.2 Casos de uso del MVP

CU-01: Primer programa — Hola Mundo

```
// اول برنامج في لغة العرب
متغير الاسم = "احمد"
اطبع("مرحبا يا " + الاسم)

// الناتج: مرحبا يا احمد
```

CU-02: Cálculo y lógica básica

```
متغير العمر = 17
متغير السنوات = 18 - العمر

(اذا (العمر =< 18) {
    اطبع("يمكنك التصويت")
} (واذا (العمر =< 16) {
    اطبع("بعد " + السنوات + " سنوات يمكنك التصويت")
} والا {
    اطبع("لا تزال صغيرا")
}
```

CU-03: Funciones y recursión

```
(دالة فيبوناتشي(ن) {
    (اذا (ن <= 1) {
        ارجع ن
    }
    (ارجع فيبوناتشي(ن - 1) + فيبوناتشي(ن - 2)
}

اطبع(فيبوناتشي(10)) // 55
```

CU-04: Programación orientada a objetos

```
صنف حساب_بنكي {
    (دالة انشاء(الاسم, الرصيد_الاولي) {
        هذا.الاسم = الاسم
        هذا.الرصيد = الرصيد_الاولي
        هذا.العمليات = []
    }

    (دالة ايداع(المبلغ) {
        (اذا (المبلغ <= 0) {
            ارمي("المبلغ يجب ان يكون اكبر من صفر")
        }
        هذا.الرصيد = هذا.الرصيد + المبلغ
        (هذا.العمليات.اضف("ايداع: " + المبلغ)
        ارجع هذا
    }

    (دالة سحب(المبلغ) {
        (اذا (المبلغ > هذا.الرصيد) {
            ارمي("رصيد غير كافٍ")
        }
        هذا.الرصيد = هذا.الرصيد - المبلغ
        ارجع هذا
```

```
    }

    دالة اطبع_كشف_الحساب() {
        اطبع("الحساب: " + هذا.الاسم)
        اطبع("الرصيد: " + هذا.الرصيد)
        لكل عملية في هذا.العمليات {
            اطبع("  - " + عملية)
        }
    }
}

متغير حسابي = جديد حساب_بنكي("احمد", 1000)
حسابي.ايداع(500).ايداع(200).سحب(100)
حسابي.اطبع_كشف_الحساب()
```

5. Especificación Completa del Lenguaje

5.1 Filosofía de diseño

El diseño de Lughat Al-Arab sigue seis principios que se aplican en orden de prioridad cuando hay conflictos:

1. Legibilidad ante todo: el código debe leerse como árabe natural, no como transliteración de C.
2. Coherencia: casos similares deben tener sintaxis similar. No hay excepciones sin justificación fuerte.
3. Explicitud sobre implicitismo: mejor ser verboso y claro que conciso y críptico. Especialmente para principiantes.
4. Errores útiles: el compilador siempre debe decir qué falló, dónde, y cómo arreglarlo — en árabe.
5. Progresividad: el lenguaje debe ser fácil de aprender para principiantes y potente para expertos.
6. Árabes primero: cuando hay una decisión de diseño ambigua, se elige la opción más cómoda para un hablante árabe nativo.

Decisiones de diseño fundamentales

Escritura RTL en archivos fuente: los archivos .عرب se escriben de derecha a izquierda. El editor (VSCode con la extensión) gestiona esto automáticamente. El compilador normaliza la dirección del texto antes del análisis.

Sin hamza ni diacríticos: las palabras como "إذا" se escriben "إرجع" ,"اذا" se escribe "ارجع". Esto elimina la ambigüedad de teclado y hace el código más uniforme. El lexer acepta ambas formas (con y sin hamza) para mayor tolerancia, pero la forma canónica es sin hamza.

Números occidentales (0-9): aunque los números indoarábigos (٠١٢٣...) son familiares para araboparlantes, los dígitos occidentales son universalmente reconocidos y evitan problemas de dirección bidireccional (bidi) en editores de texto. El lexer acepta ambos formatos y los normaliza internamente a integers estándar.

Llaves para bloques: se usa { } en lugar de indentación significativa. Razón: los editores RTL tienen comportamientos impredecibles con la indentación, y las llaves son inequívocas independientemente del modo de escritura. Además, facilitan la portabilidad del estilo de código de lenguajes como JavaScript o Rust.

Punto y coma opcional: el final de una instrucción se detecta automáticamente por salto de línea o por el inicio de un nuevo token que no puede ser continuación de la instrucción anterior. El punto y coma (;) está disponible pero no es obligatorio.

5.2 Sistema de tipos

Tipos primitivos

Tipo árabe	Tipo inglés	Tamaño	Ejemplo literal	Descripción
عدد	Number	64 bits	42, 3.14, -7	Número entero o flotante (unificado)
عدد_صحيح	Integer	64 bits	42, -100, 0	Entero con signo, 64 bits
عدد_عشري	Float	64 bits	3.14, -0.5	Punto flotante IEEE 754
نص	String	Variable	"مرحبا"	Cadena de texto Unicode UTF-8
صح	Boolean true	1 bit	صح	Valor verdadero
خطا	Boolean false	1 bit	خطا	Valor falso
فارغ	Null/None	0	فارغ	Ausencia de valor

Tipos compuestos

Tipo árabe	Equivalente	Ejemplo	Descripción
قائمة	Array/List	[1, 2, 3] أو قائمة()	Lista ordenada, tamaño dinámico
قاموس	Dict/Map	{"مفتاح": "قيمة"}	Mapa clave-valor, hash map
مجموعة	Set	3 ,2 ,1)مجموعة)	Conjunto sin duplicados
صنف	Class	صنف اسم { }	Clase con atributos y métodos
دالة	Function	دالة اسم(a, b) { }	Función de primera clase
مصفوفة	Tensor/Matrix	3 ,3)مصفوفة)	Tensor multidimensional (para IA)
نتيجة	Result<T,E>	نتيجة(قيمة)	Valor o error (manejo funcional)

Inferencia de tipos

El sistema de tipos de Lughat Al-Arab es de tipado dinámico en el MVP y optativo en versiones futuras. El tipo se infiere del valor asignado. En la versión 1.0 se añadirá tipado estático opcional con anotaciones de tipo:

```
// Tipado dinámico (MVP)
25 = متغير العمر           // tipo inferido: عدد
سارة" = متغير الاسم"       // tipo inferido: نص

// Tipado estatico opcional (v1.0)
25 = متغير العمر: عدد_صحيح
سارة" = نص :متغير الاسم"

// Parametros de funcion con tipo
```

```
دالة sumar(a: عدد, b: عدد <- (عدد {
    ارجع a + b
}
```

5.3 Palabras clave (Keywords)

La siguiente tabla contiene todos los keywords de Lughat Al-Arab v0.1. Esta lista está congelada para el MVP — no se añadirán nuevos keywords hasta la versión 0.2. La forma canónica es sin hamza y sin diacríticos.

Keyword árabe	Forma alternativa	Equivalente inglés	Categoría	Ejemplo
اذا	إذا	if	Control	اذا (س > 0) { }
واذا	وإذا	else if	Control	} واذا (س == 0) {
والا	وإلا	else	Control	} والا {
بينما	—	while	Bucle	بينما (صح) { }
لكل	—	for each	Bucle	لكل عنصر في القائمة { }
كرر	—	for (i=0..n)	Bucle	كرر من 0 حتى 10 { }
توقف	—	break	Bucle	توقف
تابع	—	continue	Bucle	تابع
دالة	—	function/fn	Definición	دالة احسب(أ، ب) { }
ارجع	أرجع	return	Función	ارجع النتيجة
صنف	—	class	OOP	صنف حيوان { }
هذا	—	this/self	OOP	"هذا.الاسم = "بسبوسة
جديد	—	new	OOP	جديد صنف(params)
يرث	—	extends	OOP	صنف قطة يرث حيوان { }
متغير	—	let/var	Variable	"متغير الاسم = "احمد
ثابت	—	const	Variable	ثابت PI = 3.14159
عدد	—	number	Tipo	متغير س: عدد = 5
نص	—	string	Tipo	"متغير ر: نص = "هي
قائمة	—	list/array	Tipo	متغير ل: قائمة = []
قاموس	—	dict/map	Tipo	متغير م: قاموس = {}
صح	—	true	Literal	متغير ت = صح
خطا	—	false	Literal	متغير ف = خطا
فارغ	—	null/none	Literal	متغير ق = فارغ
اطبع	—	print	I/O	اطبع("مرحبا")

Keyword árabe	Forma alternativa	Equivalente inglés	Categoría	Ejemplo
اقرا	اقرأ	input/read	I/O	متغير س = اقرا("ادخل اسمك :")
حاول	—	try	Error	حاول { } التقط { }
التقط	—	catch	Error	التقط (خطا) { }
اخيرا	أخيراً	finally	Error	اخيرا { }
ارمي	ارمِ	throw	Error	ارمي("رسالة الخطا")
استورد	—	import	Módulo	استورد رياضيات
صدر	—	export	Módulo	صدر دالة احسب
نموذج	—	model	IA	متغير م = نموذج(...)
درب	درّب	train	IA	درب(النموذج, البيانات)
تنبا	تنبأ	predict	IA	تنبا(النموذج, المدخل)
و	—	and/&&	Lógica	اذا (أ و ب)
او	أو	or/||	Lógica	اذا (أ او ب)
ليس	—	not/!	Lógica	اذا (ليس صح)
في	—	in	Colección	لكل س في القائمة

5.4 Sintaxis formal — Gramática EBNF

La siguiente gramática describe formalmente la sintaxis de Lughat Al-Arab v0.1. La notación es EBNF (Extended Backus-Naur Form): [ ] = opcional, { } = cero o más repeticiones, | = alternativa, " " = terminal literal.

Programa y declaraciones

```
programa        ::= declaracion*

declaracion     ::= decl_variable
                  | decl_funcion
                  | decl_clase
                  | sentencia
                  | sentencia_importar

decl_variable   ::= ("ثابت" | "متغير") IDENTIFICADOR [":" TIPO] "=" expresion

decl_funcion    ::= "دالة" IDENTIFICADOR "(" parametros ")" ["->" TIPO] bloque

parametros      ::= [ parametro { "،" parametro } ]
parametro       ::= IDENTIFICADOR [":" TIPO] ["=" expresion]
```

```
decl_clase      ::= "صنف" IDENTIFICADOR ["يرث" IDENTIFICADOR] "{" miembro*
"}"
miembro         ::= decl_variable | decl_funcion

sentencia_importar ::= "استورد" RUTA_MODULO ["ك" IDENTIFICADOR]
RUTA_MODULO     ::= IDENTIFICADOR { "." IDENTIFICADOR }
```

Sentencias

```
sentencia       ::= sent_expresion
                  | sent_si
                  | sent_bينما
                  | sent_لكل
                  | sent_كرر
                  | sent_ارجع
                  | sent_توقف
                  | sent_تابع
                  | sent_حاول
                  | sent_ارمي
                  | bloque

sent_si         ::= "اذا" "(" expresion ")" bloque
                    { "واذا" "(" expresion ")" bloque }
                    [ "والا" bloque ]

sent_بينما" =::     بينما" "(" expresion ")" bloque

sent_لكل" =::       لكل" IDENTIFICADOR "في" expresion bloque

sent_من" "كرر" =::       كرر" expresion "حتى" expresion ["بخطوة" expresion]
bloque

sent_ارجع" =::      ارجع" [ expresion ]
sent_توقف" =::      توقف"
sent_تابع" =::      تابع"

sent_حاول" =::      حاول" bloque
                    { "التقط" "(" IDENTIFICADOR [":" TIPO] ")" bloque }
                    [ "اخيرا" bloque ]

sent_ارمي" =::      ارمي" "(" expresion ")"

bloque          ::= "{" declaracion* "}"
sent_expresion ::= expresion [";"]
```

Expresiones y precedencia

```
expresion       ::= asignacion
```

```
asignacion      ::= logica_o [ ("=" | "+=" | "-=" | "*=" | "/=") asignacion ]

logica_o        ::= logica_او" } و" logica_و }
logica_و        ::= igualdad  { "و"   igualdad  }
igualdad        ::= comparacion { ("==" | "!=") comparacion }
comparacion     ::= adicion { ("<" | "<=" | ">" | ">=") adicion }
adicion         ::= mult    { ("+" | "-") mult }
mult            ::= unario  { ("*" | "/" | "%" | "**") unario }
unario          ::= [ "-" | "ليس" ] postfijo

postfijo        ::= primario
                    { "." IDENTIFICADOR                  // acceso a campo
                    | "." IDENTIFICADOR "(" argumentos ")"  // llamada a
metodo
                    | "(" argumentos ")"                // llamada a funcion
                    | "[" expresion "]"                 // indice
                    }

primario        ::= NUMERO | CADENA | "فارغ" | "خطا" | "صح"
                  | IDENTIFICADOR
                  | "(" expresion ")"
                  | lista_literal
                  | قاموس_literal
                  | "جديد" IDENTIFICADOR "(" argumentos ")"
                  | lambda

lista_literal   ::= "[" [ expresion { "،" expresion } ] "]"
قاموس_literal   ::= "{" [ par_clave { "،" par_clave } ] "}"
par_clave       ::= expresion ":" expresion

lambda          ::= "دالة" "(" parametros ")" "->" expresion

argumentos      ::= [ expresion { "،" expresion } ]

IDENTIFICADOR   ::= [\u0600-\u06FF_][\u0600-\u06FF\w_]*
NUMERO          ::= [0-9]+ ("." [0-9]+)?
CADENA          ::= "\"" [^"\\]* "\""
TIPO            ::= IDENTIFICADOR
```

5.5 Operadores y precedencia

La siguiente tabla muestra todos los operadores ordenados de mayor a menor precedencia. Los operadores con la misma precedencia son asociativos por la izquierda excepto donde se indique.

Nivel	Operadores	Asociatividad	Descripción
9 (max)	()	izquierda	Llamada a función / agrupación
8	. []	izquierda	Acceso a campo / indexado
7	** (potencia)	derecha	Exponenciación
6	- ليس (unario)	derecha	Negación aritmética y lógica

Nivel	Operadores	Asociatividad	Descripción
5	* / %	izquierda	Multiplicación, división, módulo
4	+ -	izquierda	Suma, resta / concatenación de نص
3	< <= > >= == !=	izquierda	Comparación e igualdad
2	و (AND)	izquierda	AND lógico (cortocircuito)
1	او (OR)	izquierda	OR lógico (cortocircuito)
0 (min)	= += -= *= /=	derecha	Asignación y asignación compuesta

5.6 Sistema de módulos

Los módulos en Lughat Al-Arab son archivos .عرب. La librería estándar se distribuye como módulos precompilados. El sistema de módulos sigue el modelo de Python: cada archivo es un módulo, los directorios con un archivo __عرب.__حزمة son paquetes.

```
// importar modulo completo
استورد رياضيات
اطبع(رياضيات.جذر(16)) // 4.0

// importar con alias
استورد رياضيات كـ ر
اطبع(ر.جذر(9)) // 3.0

// importar funciones especificas
من رياضيات pi, استورد جذر
اطبع(جذر(25)) // 5.0
اطبع(pi)          // 3.14159...

// exportar desde un modulo
صدر دالة احسب_مساحة(طول, عرض) {
    ارجع طول * عرض
}
صدر ثابت VERSION = "1.0.0"
```

Librería estándar v0.1 (MVP)

Módulo	Funciones incluidas	Descripción
رياضيات	جذر, اس, لوغاريتم, جيب, جيب_تمام, pi, e	Operaciones matemáticas
نصوص	طول, قطع, دمج, ابحث, استبدل, حروف_كبيرة, مقارن	Manipulación de strings
قوائم	اضف, احذف, رتب, عكس, ابحث, طول, مرشح, حول	Operaciones sobre listas
ملفات	اقرا_ملف, اكتب_ملف, وجود_ملف, احذف_ملف	I/O de archivos
شبكة	جلب, ارسل, اتصل, خادم_بسيط	Operaciones de red HTTP básicas

Módulo	Funciones incluidas	Descripción
وقت	الان, نم, قياس_الوقت	Tiempo y temporización
عشوائي	عدد_عشوائي, اختر, خلط	Generación aleatoria

5.7 Manejo de errores

El manejo de errores en Lughat Al-Arab usa el modelo try/catch clásico, llamado حاول/التقط. Adicionalmente, para código funcional, existe el tipo نتيجة que permite propagar errores sin excepciones.

```
// modelo clasico حاول / التقط
حاول {
    اقرا_ملف = ملفات.متغير البيانات("datos.csv")
    اطبع("تم تحميل " + طول(البيانات) + " سطر")
} التقط (خطا_الملف) {
    اطبع("فشل تحميل الملف: " + خطا_الملف.الرسالة)
} اخيرا {
    اطبع("انتهت عملية التحميل")
}

// lanzar excepciones personalizadas
صنف خطا_البيانات يرث خطا {
    دالة انشاء(الرسالة, رقم_السطر) {
        هذا.الرسالة = الرسالة
        هذا.السطر = رقم_السطر
    }
}

دالة تحقق_من_العمر(عمر) {
    اذا (عمر < 0) {
        ارمي(جديد خطا_البيانات("العمر لا يمكن ان يكون سالبا", 1))
    }
    ارجع عمر
}
```

5.8 Ejemplos de programas completos

Ejemplo 1: Algoritmo de ordenación burbuja

```
دالة ترتيب_فقاعي(القائمة) {
    متغير ن = طول(القائمة)
    كرر من 0 حتى ن {
        كرر من 0 حتى ن - مرة - 1 {
            اذا (القائمة[ر] > القائمة[ر + 1]) {
                متغير مؤقت = القائمة[ر]
                القائمة[ر] = القائمة[ر + 1]
```

```
                    القائمة[ر + 1] = مؤقت
                }
            }
        }
    ارجع القائمة
}

[متغير الارقام = 64, 34, 25, 12, 22, 11, 90
اطبع(ترتيب_فقاعي(الارقام))
// [11, 12, 22, 25, 34, 64, 90]
```

Ejemplo 2: Servidor HTTP mínimo

```
استورد شبكة

(متغير الخادم = شبكة.خادم_بسيط(8080)

الخادم.على("طلب", دالة(الطلب, الرد) {
    اذا (الطلب.المسار == "/") {
        الرد.ارسل(200, "مرحبا بالعالم!")
    } والا {
        الرد.ارسل(404, "الصفحة غير موجودة")
    }
})

اطبع("الخادم يعمل على المنفذ 8080")
الخادم.ابدا()
```

6. Arquitectura del Compilador

6.1 Pipeline completo

El compilador de Lughat Al-Arab sigue el pipeline clásico de compiladores, con adaptaciones para texto árabe RTL. El MVP implementa hasta el intérprete de árbol. La versión 1.0 añade generación de código LLVM IR y compilación a binario nativo.

Pipeline del compilador de Lughat Al-Arab

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    Código fuente (.عرب, UTF-8 RTL)                           │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ FASE 1 — LEXER (Análisis léxico)                                             │
│ Entrada: texto raw  →  Salida: lista de tokens                               │
│ • Normalización Unicode (NFC)                                                │
│ • Normalización bidi (RTL→LTR interno)                                       │
│ • Tokenización: keywords, identificadores, literales, operadores             │
│ • Tracking de posición (línea, columna) para mensajes de error               │
└──────────────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ FASE 2 — PARSER (Análisis sintáctico)                                        │
│ Entrada: lista de tokens  →  Salida: AST (árbol)                             │
│ • Recursive Descent Parser (RDP)                                             │
```

```
│ • Construye el árbol de sintaxis abstracta                    │
│ • Detecta errores de sintaxis con posición exacta             │
└──────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────┐
│ FASE 3 — ANALIZADOR SEMÁNTICO                                 │
│ Entrada: AST → Salida: AST anotado con tipos e información    │
│ • Resolución de nombres (symbol table)                        │
│ • Verificación de tipos (tipado dinámico en MVP)              │
│ • Detección de variables no declaradas                        │
│ • Verificación de aridad de funciones                         │
└──────────────────────────────────────────────────────────────┘
                          │
                ┌─────────┴─────────┐
                │                   │
        ┌───────▼───────┐   ┌───────▼──────────────────────┐
        │ MVP: INTÉRPRETE DE │ │ v1.0: GENERADOR DE CÓDIGO LLVM IR │
        │ ÁRBOL (Tree-walking) │ │ • Lowering de AST a LLVM IR │
        │ • Recorre AST nodo a │ │ • Optimizaciones (O1/O2/O3) │
        │   nodo y lo ejecuta │ │ • Binario nativo (x86_64 / ARM64) │
        │ • Sin compilación │ │ • Enlazado con libc y libárabe │
        └──────────────────┘   └──────────────────────────────┘
```

6.2 Lexer — Análisis léxico

Responsabilidades del Lexer

El Lexer (también llamado analizador léxico o tokenizador) es el primer componente del pipeline. Su trabajo es convertir el texto fuente bruto en una secuencia de tokens — las unidades mínimas con significado del lenguaje. El Lexer no entiende la estructura del programa, solo reconoce piezas individuales.

Manejo de texto árabe en el Lexer

El texto árabe presenta desafíos únicos que el Lexer debe gestionar antes de la tokenización:

- Normalización Unicode (NFC): el árabe puede representarse con múltiples secuencias de codepoints equivalentes. Por ejemplo, la letra "أ" puede ser U+0627 + U+0654 o directamente U+0623. El Lexer normaliza a NFC antes de procesar.
- Hamza equivalente: el Lexer acepta tanto "اذا" como "إذا" y los trata como el mismo token KEYWORD_AZA. La forma canónica interna es sin hamza.
- Dirección del texto: internamente el Lexer trabaja con la secuencia de caracteres en orden de memoria (LTR), que en árabe corresponde a derecha-a-izquierda visual. No confundir la dirección de visualización con el orden de procesamiento.
- Rangos Unicode: los identificadores árabes se detectan con el rango \u0600-\u06FF (Árabe básico) y \uFB50-\uFDFF (Árabe extendido A). Los dígitos son siempre ASCII 0-9.

Tipos de tokens

Tipo de token	Descripción	Ejemplos
KEYWORD	Palabra clave reservada	اذا, والا, دالة, متغير, صنف
IDENTIFICADOR	Nombre definido por el usuario	العمر, الاسم, احسب_مربع
NÚMERO_ENTERO	Literal numérico entero	42, 0, -7, 1000000
NÚMERO_FLOTANTE	Literal numérico con decimales	3.14, -0.5, 1.0e10
CADENA	Literal de texto entre comillas	"مرحبا", "hello"
OP_ARITMÉTICO	Operadores matemáticos	+ - * / % **
OP_COMPARACIÓN	Operadores de comparación	== != < <= > >=
OP_LÓGICO	Operadores lógicos	و او ليس
OP_ASIGNACIÓN	Asignación y compuestos	= += -= *= /=
PUNTUACIÓN	Delimitadores y separadores	( ) { } [ ] ، . ;
COMENTARIO	Comentarios de una línea	// cualquier texto hasta fin de línea
COMENTARIO_BLOQUE	Comentarios multilínea	/* texto */

Tipo de token	Descripción	Ejemplos
NUEVA_LÍNEA	Fin de línea (para ASI)	\n (newline)
FIN_ARCHIVO	Señal de fin del archivo	EOF

Implementación del Lexer en Python

El Lexer del MVP se implementa en Python 3.11+. La estructura principal es una clase Lexer con un puntero de posición que avanza por el texto carácter a carácter. El método principal es siguiente_token() que retorna el próximo token y avanza el puntero.

```
import re
import unicodedata
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class Token:
    tipo: str           # "KEYWORD", "IDENTIFICADOR", etc.
    valor: str          # el texto exacto del token
    linea: int          # numero de linea (para errores)
    columna: int        # numero de columna
    valor_canonico: str # forma normalizada (ej: "اذا" para ambas formas)

KEYWORDS = {
    "اذا": "KW_AZDA",     "إذا": "KW_AZDA",
    "والا": "KW_WALLA",   "وإلا": "KW_WALLA",
    "دالة": "KW_DALA",
    "متغير": "KW_MUTAGHAYYIR",
    "صنف": "KW_SINF",
    "هذا": "KW_HADHA",
    "ارجع": "KW_IRJAH",   "أرجع": "KW_IRJAH",
    "اطبع": "KW_ITBA",
    "لكل": "KW_LIKULL",
    "بينما": "KW_BAYNAMA",
    "صح":  "KW_SAHH",     "خطا": "KW_KHATA",
    "فارغ": "KW_FARIGH",
    # ... resto de keywords
}

class Lexer:
    def __init__(self, codigo: str):
        # Normalizar a NFC para consistencia Unicode
        self.codigo = unicodedata.normalize("NFC", codigo)
        self.pos = 0
        self.linea = 1
        self.columna = 1
        self.tokens: List[Token] = []

    def tokenizar(self) -> List[Token]:
        while self.pos < len(self.codigo):
            self._saltar_espacios()
            if self.pos >= len(self.codigo):
```

```
                break
            token = self._siguiente_token()
            if token:
                self.tokens.append(token)
        self.tokens.append(Token("FIN_ARCHIVO", "", self.linea,
self.columna, ""))
        return self.tokens

    def _siguiente_token(self) -> Optional[Token]:
        c = self.codigo[self.pos]
        # Comentario de una linea
        if c == "/" and self._peek() == "/":
            return self._leer_comentario_linea()
        # Numero
        if c.isdigit():
            return self._leer_numero()
        # Cadena
        if c == chr(34):  # comilla doble
            return self._leer_cadena()
        # Caracter arabe o subrayado -> identificador o keyword
        if self._es_inicio_identificador(c):
            return self._leer_identificador()
        # Operadores y puntuacion
        return self._leer_operador_o_puntuacion()

    def _es_inicio_identificador(self, c: str) -> bool:
        cp = ord(c)
        return (0x0600 <= cp <= 0x06FF or   # Arabe basico
                0xFB50 <= cp <= 0xFDFF or   # Arabe extendido A
                c == "_" or c.isalpha())    # ASCII o unicode general

    def _leer_identificador(self) -> Token:
        inicio = self.pos
        col_inicio = self.columna
        while self.pos < len(self.codigo):
            c = self.codigo[self.pos]
            if not (self._es_inicio_identificador(c) or c.isdigit() or c ==
"_"):
                break
            self._avanzar()
        texto = self.codigo[inicio:self.pos]
        tipo = KEYWORDS.get(texto, "IDENTIFICADOR")
        canonico = texto  # simplificado; en produccion: normalizar hamza
        return Token(tipo, texto, self.linea, col_inicio, canonico)
```

6.3 Parser — Análisis sintáctico

Elección del algoritmo: Recursive Descent Parser

El Parser convierte la lista de tokens en un Árbol de Sintaxis Abstracta (AST). Para el MVP se implementa un Recursive Descent Parser (RDP), también llamado parser LL(1) con backtracking limitado. Esta elección se justifica por tres razones:

- Legibilidad: el código del parser refleja directamente la gramática EBNF. Cada regla gramatical se convierte en una función Python. Es la implementación más fácil de depurar y modificar.
- Control de errores: el RDP permite generar mensajes de error muy específicos porque el parser sabe exactamente qué estaba esperando en cada punto.
- Velocidad de implementación: un RDP completo para Lughat Al-Arab se puede escribir en ~600 líneas de Python, frente a las miles de líneas que requeriría configurar un generador de parsers como PLY o ANTLR.

La alternativa considerada fue usar LALR(1) con PLY (Python Lex-Yacc), que es más potente para gramáticas ambiguas, pero su curva de aprendizaje y los mensajes de error genéricos lo hacen subóptimo para este proyecto.

Estructura del Parser

```
class Parser:
    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.pos = 0

    @property
    def token_actual(self) -> Token:
        return self.tokens[self.pos]

    def consumir(self, tipo_esperado: str) -> Token:
        """Consume el token actual si es del tipo esperado, sino lanza
error."""
        if self.token_actual.tipo != tipo_esperado:
            raise ErrorSintaxis(
                f"خطا في التركيب: توقعت {tipo_esperado} "
                f"لكن وجدت {self.token_actual.tipo!r} "
                f"في السطر {self.token_actual.linea}، العمود
{self.token_actual.columna}"
            )
        token = self.token_actual
        self.pos += 1
        return token

    def parsear_programa(self) -> NodoPrograma:
        """Punto de entrada: parsea el programa completo."""
        declaraciones = []
        while self.token_actual.tipo != "FIN_ARCHIVO":
            declaraciones.append(self.parsear_declaracion())
        return NodoPrograma(declaraciones)

    def parsear_declaracion(self) -> NodoAST:
        t = self.token_actual.tipo
        if t == "KW_MUTAGHAYYIR": return self.parsear_decl_variable()
        if t == "KW_DALA":        return self.parsear_decl_funcion()
        if t == "KW_SINF":        return self.parsear_decl_clase()
```

```
        if t == "KW_AZDA":        return self.parsear_sent_si()
        if t == "KW_BAYNAMA":     return self.parsear_sent_mientras()
        if t == "KW_LIKULL":      return self.parsear_sent_para_cada()
        if t == "KW_IRJAH":       return self.parsear_sent_retornar()
        if t == "KW_HAWAL":       return self.parsear_sent_intentar()
        if t == "KW_ARMEE":       return self.parsear_sent_lanzar()
        return self.parsear_sent_expresion()

    def parsear_decl_variable(self) -> NodoDeclVariable:
        es_constante = (self.token_actual.tipo == "KW_THABIT")
        self.consumir(self.token_actual.tipo)  # متغير o ثابت
        nombre = self.consumir("IDENTIFICADOR").valor
        tipo_anotado = None
        if self.token_actual.valor == ":":
            self.consumir("PUNTUACION")
            tipo_anotado = self.consumir("IDENTIFICADOR").valor
        self.consumir("OP_ASIGNACION")  # =
        valor = self.parsear_expresion()
        return NodoDeclVariable(nombre, valor, tipo_anotado, es_constante)

    def parsear_decl_funcion(self) -> NodoDeclFuncion:
        self.consumir("KW_DALA")      # دالة
        nombre = self.consumir("IDENTIFICADOR").valor
        self.consumir("PUNTUACION")   # (
        params = self.parsear_parametros()
        self.consumir("PUNTUACION")   # )
        tipo_retorno = None
        if self.token_actual.valor == "->":
            self.consumir("OP_FLECHA")
            tipo_retorno = self.consumir("IDENTIFICADOR").valor
        cuerpo = self.parsear_bloque()
        return NodoDeclFuncion(nombre, params, cuerpo, tipo_retorno)
```

6.4 AST — Árbol de Sintaxis Abstracta

El AST (Abstract Syntax Tree) es la representación intermedia central del compilador. Es un árbol donde cada nodo representa una construcción del lenguaje. El analizador semántico y el intérprete/generador de código operan sobre el AST, no sobre el texto fuente ni sobre los tokens.

Jerarquía de nodos del AST

```
from dataclasses import dataclass, field
from typing import List, Optional, Any

# Clase base para todos los nodos
@dataclass
class NodoAST:
    linea: int = 0
    columna: int = 0
```

```
# ─── Nodos de declaracion ──────────────────────────────────────────
@dataclass
class NodoPrograma(NodoAST):
    cuerpo: List[NodoAST] = field(default_factory=list)

@dataclass
class NodoDeclVariable(NodoAST):
    nombre: str = ""
    valor: Optional[NodoAST] = None
    tipo_anotado: Optional[str] = None
    es_constante: bool = False

@dataclass
class NodoDeclFuncion(NodoAST):
    nombre: str = ""
    parametros: List["NodoParametro"] = field(default_factory=list)
    cuerpo: "NodoBloque" = None
    tipo_retorno: Optional[str] = None

@dataclass
class NodoParametro(NodoAST):
    nombre: str = ""
    tipo: Optional[str] = None
    valor_defecto: Optional[NodoAST] = None

@dataclass
class NodoDeclClase(NodoAST):
    nombre: str = ""
    padre: Optional[str] = None  # herencia
    miembros: List[NodoAST] = field(default_factory=list)

# ─── Nodos de sentencia ────────────────────────────────────────────
@dataclass
class NodoBloque(NodoAST):
    sentencias: List[NodoAST] = field(default_factory=list)

@dataclass
class NodoSi(NodoAST):
    condicion: NodoAST = None
    entonces: NodoBloque = None
    ramas_elif: List[tuple] = field(default_factory=list)  # [(cond,
bloque)]
    sino: Optional[NodoBloque] = None

@dataclass
class NodoBucle_بينما(NodoAST):
    condicion: NodoAST = None
    cuerpo: NodoBloque = None

@dataclass
class NodoBucle_لكل(NodoAST):
    variable: str = ""
    iterable: NodoAST = None
    cuerpo: NodoBloque = None

@dataclass
```

```
class NodoRetornar(NodoAST):
    valor: Optional[NodoAST] = None

@dataclass
class NodoLanzar(NodoAST):
    excepcion: NodoAST = None

@dataclass
class NodoIntentar(NodoAST):
    cuerpo: NodoBloque = None
    capturas: List[tuple] = field(default_factory=list)  # [(nombre, tipo,
bloque)]
    finalmente: Optional[NodoBloque] = None

# ─── Nodos de expresion ──────────────────────────────────────────
@dataclass
class NodoAsignacion(NodoAST):
    objetivo: NodoAST = None
    valor: NodoAST = None
    operador: str = "="  # "=", "+=", "-=", etc.

@dataclass
class NodoOpBinaria(NodoAST):
    izquierda: NodoAST = None
    operador: str = ""
    derecha: NodoAST = None

@dataclass
class NodoOpUnaria(NodoAST):
    operador: str = ""
    operando: NodoAST = None

@dataclass
class NodoLlamada(NodoAST):
    funcion: NodoAST = None
    argumentos: List[NodoAST] = field(default_factory=list)

@dataclass
class NodoAcceso(NodoAST):
    objeto: NodoAST = None
    campo: str = ""

@dataclass
class NodoIndice(NodoAST):
    objeto: NodoAST = None
    indice: NodoAST = None

@dataclass
class NodoNuevo(NodoAST):
    clase: str = ""
    argumentos: List[NodoAST] = field(default_factory=list)

# ─── Nodos literales ─────────────────────────────────────────────
@dataclass
class NodoLiteralNumero(NodoAST):
    valor: float = 0.0
```

```
@dataclass
class NodoLiteralCadena(NodoAST):
    valor: str = ""

@dataclass
class NodoLiteralBool(NodoAST):
    valor: bool = False

@dataclass
class NodoLiteralNulo(NodoAST):
    pass

@dataclass
class NodoLiteralLista(NodoAST):
    elementos: List[NodoAST] = field(default_factory=list)

@dataclass
class NodoLiteralDict(NodoAST):
    pares: List[tuple] = field(default_factory=list)  # [(clave, valor)]

@dataclass
class NodoIdentificador(NodoAST):
    nombre: str = ""
```

6.5 Analizador semántico

El analizador semántico recorre el AST y verifica que el programa sea semánticamente correcto — es decir, que más allá de la sintaxis, tenga sentido. Para el MVP se implementan las siguientes verificaciones:

Symbol table (tabla de símbolos)

El analizador mantiene una tabla de símbolos con alcance (scope). Cada bloque { } crea un nuevo scope. Al salir del bloque, los símbolos de ese scope se eliminan. Los scopes son anidados: un scope hijo puede acceder a los símbolos del scope padre, pero no al revés.

```
class Scope:
    def __init__(self, padre=None):
        self.simbolos = {}  # nombre -> {tipo, es_constante, ...}
        self.padre = padre

    def declarar(self, nombre, info):
        if nombre in self.simbolos:
            raise ErrorSemantico(f"خطا: "{nombre}" مَعرّف مسبقا في هذا النطاق")
        self.simbolos[nombre] = info

    def buscar(self, nombre):
        if nombre in self.simbolos:
```

```
            return self.simbolos[nombre]
        if self.padre:
            return self.padre.buscar(nombre)
        raise ErrorSemantico(f"خطأ: المتغير "{nombre}" غير مُعرَّف")

class AnalizadorSemantico:
    def __init__(self):
        self.scope_global = Scope()
        self.scope_actual = self.scope_global

    def _nuevo_scope(self):
        self.scope_actual = Scope(padre=self.scope_actual)

    def _cerrar_scope(self):
        self.scope_actual = self.scope_actual.padre

    def analizar(self, nodo: NodoAST):
        metodo = f"analizar_{type(nodo).__name__}"
        return getattr(self, metodo, self.analizar_generico)(nodo)
```

6.6 Intérprete de árbol (MVP)

El intérprete de árbol (tree-walking interpreter) es el corazón del MVP. Recorre el AST nodo por nodo y ejecuta directamente cada nodo. Es el enfoque más simple de implementar y suficiente para validar el diseño del lenguaje. El rendimiento no es óptimo (Python ejecutando Python), pero es completamente funcional para programas educativos y prototipado.

```
class Interprete:
    def __init__(self):
        self.entorno_global = Entorno()
        self.entorno_actual = self.entorno_global
        self._registrar_stdlib()

    def ejecutar(self, nodo: NodoAST):
        metodo = f"ejecutar_{type(nodo).__name__}"
        return getattr(self, metodo)(nodo)

    def ejecutar_NodoPrograma(self, nodo: NodoPrograma):
        resultado = None
        for sentencia in nodo.cuerpo:
            resultado = self.ejecutar(sentencia)
        return resultado

    def ejecutar_NodoDeclVariable(self, nodo: NodoDeclVariable):
        valor = self.ejecutar(nodo.valor) if nodo.valor else None
        self.entorno_actual.definir(nodo.nombre, valor, nodo.es_constante)

    def ejecutar_NodoOpBinaria(self, nodo: NodoOpBinaria):
        izq = self.ejecutar(nodo.izquierda)
        der = self.ejecutar(nodo.derecha)
        op = nodo.operador
```

```
        if op == "+":
            if isinstance(izq, str) or isinstance(der, str):
                return str(izq) + str(der)  # concatenacion
            return izq + der
        if op == "-": return izq - der
        if op == "*": return izq * der
        if op == "/":
            if der == 0:
                raise ErrorEjecucion("خطا: القسمة على صفر غير مسموح بها")
            return izq / der
        if op == "%": return izq % der
        if op == "**": return izq ** der
        if op == "==": return izq == der
        if op == "!=": return izq != der
        if op == "<":  return izq < der
        if op == "<=": return izq <= der
        if op == ">":  return izq > der
        if op == ">=": return izq >= der
        raise ErrorEjecucion(f"مشغّل غير معروف: {op}")

    def ejecutar_NodoDeclFuncion(self, nodo: NodoDeclFuncion):
        funcion = FuncionDefinidaUsuario(nodo, self.entorno_actual)
        self.entorno_actual.definir(nodo.nombre, funcion)

    def ejecutar_NodoLlamada(self, nodo: NodoLlamada):
        funcion = self.ejecutar(nodo.funcion)
        args = [self.ejecutar(a) for a in nodo.argumentos]
        if callable(funcion):
            return funcion(*args)  # función nativa
        if isinstance(funcion, FuncionDefinidaUsuario):
            return self._llamar_funcion(funcion, args)
        raise ErrorEjecucion(f"هذا ليس دالة: {funcion}")

    def _llamar_funcion(self, fn, args):
        if len(args) != len(fn.nodo.parametros):
            raise ErrorEjecucion(
                f"الدالة {fn.nodo.nombre!r} تحتاج {len(fn.nodo.parametros)} "
                f"معاملات لكن أعطيت {len(args)}"
            )
        entorno_funcion = Entorno(padre=fn.entorno_cierre)
        for param, valor in zip(fn.nodo.parametros, args):
            entorno_funcion.definir(param.nombre, valor)
        entorno_previo = self.entorno_actual
        self.entorno_actual = entorno_funcion
        try:
            self.ejecutar(fn.nodo.cuerpo)
        except SeñalRetornar as ret:
            return ret.valor
        finally:
            self.entorno_actual = entorno_previo
        return None
```

6.7 Gestión de memoria

En el MVP (intérprete Python), la gestión de memoria la realiza Python automáticamente mediante garbage collection. Para la versión 1.0 con compilación nativa en Rust, se diseñará un sistema de ownership simplificado.

Modelo de ownership del MVP

El intérprete del MVP usa semántica de valor para tipos primitivos (números, booleanos) y semántica de referencia con conteo de referencias implícito para objetos compuestos (listas, diccionarios, instancias de clase). Python gestiona el ciclo de vida de todos los objetos.

Modelo de memoria para v1.0 (Rust/LLVM)

La versión compilada usará un modelo inspirado en Rust pero simplificado para ser accesible a programadores araboparlantes:

- Cada valor tiene exactamente un propietario (دالة مالك — función propietaria).
- Cuando el propietario sale del scope, el valor se libera automáticamente (RAII).
- Las referencias se expresan con & (مرجع). Una referencia no puede sobrevivir a su propietario.
- Para el MVP no hay que preocuparse de esto — Python gestiona todo.

6.8 Mensajes de error en árabe

Todos los mensajes de error del compilador e intérprete están en árabe. Cada mensaje sigue el formato: Tipo de error + Ubicación + Descripción + Sugerencia.

Código error	Situación	Mensaje en árabe
E001	Variable no declarada	خطا E001: المتغير "س" غير معرّف في السطر 5 — هل نسيت كتابة "متغير س = ...؟"
E002	Tipo incompatible	خطا E002: لا يمكن جمع عدد مع نص في السطر 7 — استخدم تحويل_نص(س) لتحويل العدد الى نص
E003	División por cero	خطا E003: القسمة على صفر غير مسموح بها في السطر 12
E004	Aridad incorrecta	خطا E004: الدالة "احسب" تحتاج 2 معاملات لكن أُعطيت 3 في السطر 9
E005	Variable constante	خطا E005: في السطر 15 — استخدم متغير بدلا "PI" لا يمكن تغيير قيمة الثابت من ثابت
E006	Índice fuera de rango	خطا E006: الفهرس 5 خارج نطاق القائمة التي طولها 3 في السطر 18
E007	Llamada a no-función	خطا E007: "العمر" ليس دالة، لا يمكن استدعاؤه في السطر 20
E008	Herencia de clase inexistente	خطا E008: الصنف "حيوان" غير معرّف، لا يمكن الوراثة منه في السطر 3

Código error	Situación	Mensaje en árabe
E009	String sin cerrar	خطا E009: نقصت علامة اقتباس الإغلاق — السطر 6 في مكتمل غير نص "
E010	Llave sin cerrar	خطا E010: نقصت قوس الإغلاق — السطر 10 في مكتملة غير كتلة الكود }
W001	Variable declarada no usada	تحذير W001: المتغير "المجموع" مُعرَّف لكن لم يُستخدم في السطر 4
W002	Función sin return	تحذير W002: الدالة "احسب" لا تُرجع قيمة في جميع المسارات

7. IDE — Extensión de Visual Studio Code

7.1 Arquitectura de la extensión

La decisión de usar VSCode como base del IDE es estratégica: VSCode tiene ~70% de cuota de mercado entre desarrolladores (StackOverflow 2024), su núcleo Monaco Editor es open source y altamente configurable, y su API de extensiones permite implementar soporte RTL, syntax highlighting, diagnósticos en tiempo real y debugging sin modificar el editor base.

La extensión se desarrolla en TypeScript y se publica en el Marketplace de VSCode. Al publicarla, cualquier persona que tenga VSCode instalado puede añadir soporte para Lughat Al-Arab con un clic.

Estructura del proyecto de extensión

```
lughat-al-arab-vscode/
├── package.json           # Manifiesto de la extensión (punto de entrada)
├── tsconfig.json          # Configuración TypeScript
├── src/
│   ├── extension.ts       # Punto de entrada de la extensión
│   ├── lsp/
│   │   ├── client.ts      # Cliente LSP (se conecta al servidor)
│   │   └── server.ts      # Servidor LSP (analiza el codigo)
│   ├── highlighting/
│   │   └── arabic.tmLanguage.json  # Gramática TextMate
│   ├── rtl/
│   │   └── rtlSupport.ts  # Fuerza RTL en Monaco Editor
│   └── commands/
│       └── runFile.ts     # Comando "ejecutar archivo .عرب"
├── syntaxes/
│   └── arabic.tmLanguage.json
├── language-configuration.json  # Brackets, comentarios, indentacion
└── themes/
    └── arabic-dark.json   # Tema visual RTL
```

package.json — Manifiesto de la extensión

```
{
  "name": "lughat-al-arab",
  "displayName": "لغة العرب",
  "description": "Soporte para el lenguaje de programacion arabe",
  "version": "0.1.0",
  "engines": { "vscode": "^1.85.0" },
  "categories": ["Programming Languages"],
  "contributes": {
    "languages": [{
      "id": "lughat-al-arab",
```

```
      "aliases": ["Arabic Lang", "عرب"],
      "extensions": [".عرب"],
      "configuration": "./language-configuration.json"
    }],
    "grammars": [{
      "language": "lughat-al-arab",
      "scopeName": "source.arab",
      "path": "./syntaxes/arabic.tmLanguage.json"
    }],
    "commands": [
      {
        "command": "lughat-al-arab.run",
        "title": "تشغيل الملف",
        "icon": "$(play)"
      }
    ],
    "keybindings": [{
      "command": "lughat-al-arab.run",
      "key": "ctrl+shift+enter",
      "when": "editorLangId == lughat-al-arab"
    }]
  },
  "activationEvents": ["onLanguage:lughat-al-arab"]
}
```

7.2 Soporte RTL en Monaco Editor

Monaco Editor (el núcleo de VSCode) tiene soporte limitado para RTL por defecto. La extensión aplica las siguientes configuraciones para garantizar una experiencia de escritura árabe correcta:

```
// src/rtl/rtlSupport.ts
import * as vscode from "vscode";

export function activarRTL(context: vscode.ExtensionContext) {
  // Aplicar configuración RTL al abrir cualquier .عرب
  vscode.workspace.onDidOpenTextDocument((doc) => {
    if (doc.languageId === "lughat-al-arab") {
      vscode.workspace.getConfiguration("editor", doc.uri).update(
        "renderWhitespace", "none",
vscode.ConfigurationTarget.WorkspaceFolder
      );
    }
  });

  // Comando para alternar dirección del editor
  context.subscriptions.push(
    vscode.commands.registerCommand("lughat-al-arab.toggleRTL", () => {
      const editor = vscode.window.activeTextEditor;
      if (!editor) return;
      // Monaco no expone direction directamente; lo hacemos via CSS
injection
```

```
      vscode.window.showInformationMessage("وضع RTL مفعّل");
    })
  );
}

// language-configuration.json
{
  "comments": {
    "lineComment": "//"
  },
  "brackets": [
    ["{", "}"],
    ["[", "]"],
    ["(", ")"]
  ],
  "autoClosingPairs": [
    { "open": "{", "close": "}" },
    { "open": "[", "close": "]" },
    { "open": "(", "close": ")" },
    { "open": "\"", "close": "\"" }
  ],
  "surroundingPairs": [
    ["{", "}"],
    ["[", "]"],
    ["(", ")"],
    ["\"", "\""]
  ],
  "wordPattern": "[\\u0600-\\u06FF_\\w]+"
}
```

7.3 Language Server Protocol (LSP)

El LSP (Language Server Protocol) es el protocolo estándar de la industria para proveer funcionalidades de IDE de forma independiente al editor. VSCode, Neovim, Emacs y otros editores pueden usar el mismo servidor LSP. El servidor LSP de Lughat Al-Arab se implementa en Python y se comunica con la extensión VSCode a través de JSON-RPC sobre stdio.

Funcionalidades del LSP en el MVP

Funcionalidad LSP	Descripción	Prioridad MVP
textDocument/publishDiagnostics	Mostrar errores y advertencias subrayados en rojo/amarillo	ALTA
textDocument/completion	Autocompletado de keywords y variables declaradas	ALTA
textDocument/hover	Mostrar tipo y descripción al pasar el ratón sobre un símbolo	MEDIA
textDocument/definition	Ir a la definición de una función o variable (Ctrl+Click)	MEDIA

Funcionalidad LSP	Descripción	Prioridad MVP
textDocument/rename	Renombrar una variable en todos sus usos	BAJA
textDocument/formatting	Formatear el código automáticamente	BAJA
textDocument/semanticTokens	Resaltado semántico (no solo sintáctico)	MEDIA

7.4 Syntax highlighting con TextMate

El syntax highlighting se define con una gramática TextMate (archivo .tmLanguage.json). VSCode usa este formato para colorear el código en tiempo real sin necesidad del servidor LSP. La gramática define patrones regex que identifican cada tipo de token y les asigna un "scope" que el tema de color del editor convierte en colores.

```
{
  "name": "لغة العرب",
  "scopeName": "source.arab",
  "patterns": [
    { "include": "#comments" },
    { "include": "#strings" },
    { "include": "#keywords" },
    { "include": "#numbers" },
    { "include": "#identifiers" }
  ],
  "repository": {
    "comments": {
      "match": "//.*$",
      "name": "comment.line.double-slash.arab"
    },
    "strings": {
      "match": "\"[^\"\\\\]*\"",
      "name": "string.quoted.double.arab"
    },
    "keywords": {
      "patterns": [
        {
          "match":
"\\b(اذا | إذا | والا | وإلا | واذا | وإذا | دالة | متغير | ثابت | صنف | هذا | جديد | يرث | ارجع | أرجع | لكل | بينما | كرر | توقف | تابع | اط
بع | اقرا | اقرأ | حاول | التقط | اخيرا | أخيرا | ارمي | ارمِ | استورد | صدر | نموذج | درب | درّب | تنبا | تنبأ | صح | خطا | فارغ | و | او | أو |
ليس | في | من | حتى | بخطوة | جديد | كـ)\\b",
          "name": "keyword.control.arab"
        }
      ]
    },
    "numbers": {
      "match": "\\b[0-9]+(\\.[0-9]+)?\\b",
      "name": "constant.numeric.arab"
    },
    "identifiers": {
      "match": "[\\u0600-\\u06FF_][\\u0600-\\u06FF\\w_]*",
```

```
        "name": "variable.other.arab"
      }
    }
  }
```

8. Stack Tecnológico y Decisiones de Diseño

8.1 Resumen del stack

Componente	Tecnología elegida	Versión mínima	Justificación	Alternativa descartada
Compilador MVP	Python	3.11+	Prototipado rápido, unicode nativo, REPL fácil	Go, JavaScript
Compilador v1.0	Rust	1.75+	Velocidad nativa, seguridad de memoria, LLVM bindings	C++, Zig
Backend codegen	LLVM	17+	Estándar industria, optimizaciones probadas	Cranelift, QBE
Parser	RDP manual	—	Legible, errores precisos, fácil mantener	PLY, ANTLR, Lark
IDE	VSCode extensión	1.85+	Base instalada enorme, Monaco Editor maduro	IDE propio, Zed
LSP servidor	Python (pygls)	1.0+	Usa el mismo código del compilador, menos duplicación	TypeScript
Syntax HL	TextMate grammar	—	Compatible con VSCode, GitHub, Zed, Neovim	Tree-sitter (complejo)
Terminal embed	VSCode built-in	—	Ya incluida en VSCode, cero trabajo extra	xterm.js
Tests compilador	pytest	7.0+	Maduro, plugins para cobertura, assertions claras	unittest
CI/CD	GitHub Actions	—	Gratis, integrado con GitHub, fácil YAML	GitLab CI
Formato archivo	.عرب (UTF-8)	—	Extensión árabe nativa, reconocible	.arab (ASCII)

8.2 Por qué Python para el MVP del compilador

La elección de Python para el compilador del MVP (y no Rust directamente) se basa en un análisis de velocidad de iteración vs rendimiento:

- Velocidad de iteración: en Python se puede implementar y probar un cambio en el diseño del lenguaje en minutos. En Rust, el ciclo compile-test es más lento y los errores del borrow checker añaden fricción innecesaria cuando el diseño del lenguaje aún no está estabilizado.
- Manejo de Unicode: Python 3 maneja Unicode internamente de forma transparente. El módulo unicodedata ofrece normalización NFC/NFD con una línea de código. En Rust se necesita la crate unicode-normalization y gestión manual.

- Debugging: cuando el compilador genera un error inesperado, en Python se puede añadir un print() y ejecutar en 2 segundos. En Rust hay que recompilar (~10-30 segundos por cambio pequeño).
- Portabilidad: el compilador en Python corre en Windows, macOS y Linux sin cambios.

El plan de migración a Rust es: una vez que la especificación del lenguaje esté congelada (semana 8) y los 50 programas de test pasen, se porta el lexer y el parser a Rust. El AST se rediseña como un enum de Rust para mayor eficiencia. El intérprete se sustituye por generación de LLVM IR.

8.3 Por qué VSCode y no un IDE propio

Construir un IDE completo desde cero requeriría entre 6 y 18 meses de trabajo de un equipo completo. VSCode ofrece gratuitamente: editor de texto avanzado (Monaco), terminal integrada, gestión de archivos, debugging, extensiones, soporte para múltiples lenguajes, y una base de usuarios de millones de desarrolladores.

La API de extensiones de VSCode está específicamente diseñada para añadir soporte de nuevos lenguajes. Una extensión básica (syntax highlighting + ejecutar archivo) se puede hacer en 2-3 días. Una extensión completa con LSP (autocompletado, errores en tiempo real) toma 2-3 semanas.

8.4 Decisiones de diseño del lenguaje: justificaciones técnicas

¿Por qué llaves {} y no indentación significativa?

Python usa indentación significativa con gran éxito. Sin embargo, para Lughat Al-Arab las llaves son superiores por una razón específica: el texto árabe en editores tiene comportamientos de indentación impredecibles en modo RTL. Algunos editores añaden el espacio de indentación a la derecha (correcto para RTL) y otros a la izquierda (incorrecto). Las llaves son inequívocas independientemente del comportamiento del editor y del modo de escritura.

¿Por qué un punto y coma opcional y no obligatorio?

El punto y coma obligatorio (como en C o Java) añade ruido visual sin aportar información. El salto de línea es suficiente para delimitar instrucciones en el 99% de los casos. La inserción automática de punto y coma (ASI, como en JavaScript) se implementa en el lexer: al encontrar un salto de línea, si el token anterior puede ser el final de una instrucción, se inserta automáticamente un token PUNTO_Y_COMA implícito.

¿Por qué tipado dinámico en el MVP?

El tipado dinámico permite escribir código más conciso y reduce la fricción para principiantes. No tener que declarar tipos permite al estudiante centrarse en la lógica del programa. El tipado estático opcional se añade en v1.0 para código de producción donde el rendimiento y la corrección son críticos.

9. Roadmap de 12 Semanas

Fase 1 — Fundamentos del lenguaje (Semanas 1-4)

El objetivo de la Fase 1 es tener el primer programa ejecutable: اطبع("مرحبا") funcionando end-to-end desde el texto fuente hasta la salida en terminal.

Semana	Área	Tarea detallada	Entregable	Horas est.
S1	Diseño	Escribir gramática EBNF completa del lenguaje v0.1. Validar con 10 ejemplos a mano. Congelar lista de 35 keywords.	grammar.ebnf + keywords.md	8h
S1	Diseño	Crear repositorio GitHub con estructura de carpetas: /compiler /ide /examples /docs /tests	Repo público vacío	2h
S1	Diseño	Escribir 10 programas de ejemplo a mano (sin ejecutar). Esto valida que la sintaxis es cómoda.	10 archivos .عرب de ejemplo	4h
S2	Lexer	Implementar la clase Lexer en Python con soporte para: keywords árabes, identificadores \u0600-\u06FF, números, strings, comentarios, operadores.	lexer.py funcional	10h
S2	Tests	Suite de tests del Lexer: al menos 50 tests cubriendo todos los tipos de tokens, incluyendo casos límite (hamza, caracteres especiales, strings vacíos).	tests/test_lexer.py (50+ tests)	5h
S3	Parser	Implementar las clases de nodos AST en Python (dataclasses). Implementar el Recursive Descent Parser para declaraciones de variables y funciones.	ast_nodes.py + parser.py (parcial)	12h
S3	Parser	Completar el parser con soporte para: if/else, while, for-each, try/catch, clases, operadores aritméticos y lógicos.	parser.py completo	10h
S4	Semántico	Implementar el analizador semántico básico: symbol table con scopes, resolución de nombres, detección de variables no declaradas.	semantic.py	8h
S4	Intérprete	Implementar el intérprete de árbol para: literales, operaciones	interpreter.py (parcial)	8h

Semana	Área	Tarea detallada	Entregable	Horas est.
		binarias, variables, aطبع. Primer programa ejecutable: اطبع("مرحبا").		
S4	Tests	Test de integración end-to-end: compilar y ejecutar los 10 programas de ejemplo. Todos deben pasar.	10 tests de integración pasando	4h

Fase 2 — Intérprete completo (Semanas 5-8)

El objetivo de la Fase 2 es un intérprete completo capaz de ejecutar programas reales: clases, herencia, manejo de errores, librería estándar y 50 programas de test pasando.

Semana	Área	Tarea detallada	Entregable	Horas est.
S5	Intérprete	Completar el intérprete: funciones (closure, recursión), clases (instanciación, métodos, هذا), herencia (يرث), jديد.	interpreter.py completo	12h
S5	Intérprete	Implementar tipos compuestos: قائمة (list), قاموس (dict), مجموعة (set) con sus operaciones básicas.	Soporte tipos compuestos	6h
S6	Stdlib	Implementar módulo رياضيات: جذر, جيب, جيب_تمام, لوغاريتم, اسس, pi, e, دور (round), قيمة_مطلقة.	stdlib/رياضيات.py	4h
S6	Stdlib	Implementar módulo نصوص: طول, قطع, دمج, ابحث, استبدل, يبدأ_بـ, ينتهي_بـ, مقسم (split).	stdlib/نصوص.py	5h
S6	Stdlib	Implementar módulo قوائم: اضف, احذف, رتب, عكس, ابحث, طول, مرشح, حول (map/filter/reduce árabe).	stdlib/قوائم.py	5h
S6	Stdlib	Implementar módulo ملفات: اقرا_ملف, اكتب_ملف, وجود_ملف. Módulo وقت: الان, نم, قياس_الوقت.	stdlib/ملفات.py + وقت.py	4h
S7	Errores	Implementar el sistema de mensajes de error en árabe: al menos 25 tipos de error (E001-E025), con posición, descripción y sugerencia.	errors.py con 25 msgs	6h
S7	Intérprete	Implementar حاول/التقط/اخيرا/ارمي. Errores personalizados con صنف يرث خطا. Propagación de excepciones.	Manejo de errores completo	6h
S7	Ejemplos	Escribir 30 programas de ejemplo: Fibonacci, Torres de	30 archivos .عرب	8h

Semana	Área	Tarea detallada	Entregable	Horas est.
		Hanoi, ordenación, búsqueda binaria, calculadora, gestor de tareas.		
S8	Tests	Suite de 50 tests de integración. Cada test: código fuente + salida esperada. Todos deben pasar sin errores.	50 tests pasando al 100%	10h
S8	CLI	Implementar la interfaz de línea de comandos: لغة_العرب main.عرب para ejecutar, لغة_العرب --repl para modo interactivo.	CLI funcional	4h
S8	Buffer	Semana de buffer: corregir bugs encontrados en los tests, documentar el código del compilador, escribir README.	Bugs corregidos + docs	8h

Fase 3 — IDE en VSCode + Pulido (Semanas 9-12)

El objetivo de la Fase 3 es publicar la extensión VSCode y hacer la demo pública. El compilador debe estar estable y la documentación completa.

Semana	Área	Tarea detallada	Entregable	Horas est.
S9	VSCode	Crear el proyecto de extensión con yeoman: npm install -g yo generator-code && yo code. Configurar package.json con la extensión .عرب.	Proyecto extensión creado	3h
S9	Syntax HL	Escribir la gramática TextMate (arabic.tmLanguage.json) para keywords, strings, números, comentarios, identificadores.	Syntax highlighting funcionando	5h
S9	VSCode	Configurar language-configuration.json: brackets, autoclose, wordPattern árabe. Implementar comando "تشغيل الملف" (Ctrl+Shift+Enter).	Ejecutar .عرب desde VSCode	4h
S10	LSP	Implementar el servidor LSP en Python con pygls. Conectar con el compilador para diagnósticos en tiempo real (textDocument/publishDiagnostics).	Errores subrayados en rojo	10h
S10	LSP	Añadir autocompletado básico al LSP: keywords y variables	Autocompletado de keywords	6h

Semana	Área	Tarea detallada	Entregable	Horas est.
		declaradas en el scope actual (textDocument/completion).		
S10	RTL	Configurar Monaco Editor para modo RTL. Asegurar que el cursor, la selección y el movimiento de texto funcionen correctamente en árabe.	Editor RTL funcional	4h
S11	Docs	Escribir la documentación completa del lenguaje en árabe: referencia de todos los keywords, tipos, operadores, stdlib y ejemplos.	docs/referencia.md (árabe)	10h
S11	Docs	Escribir guía de instalación: cómo instalar Python, cómo instalar la extensión VSCode, cómo ejecutar el primer programa.	docs/instalacion.md	3h
S11	Publicación	Publicar la extensión en el VSCode Marketplace. Crear página en GitHub con README, screenshots y ejemplos.	Extensión publicada	4h
S12	Demo	Grabar video demo de 5 minutos: instalar extensión, escribir programa, ejecutar, ver errores en árabe, debugging básico.	Video demo en YouTube	4h
S12	Release	Crear GitHub Release v0.1.0: tag, changelog, binarios del compilador para Windows/macOS/Linux.	Release v0.1.0 publicada	3h
S12	Buffer	Semana de buffer: bugs críticos, feedback de primeros usuarios, correcciones de documentación.	Proyecto estabilizado	8h

10. Hitos y Métricas de Éxito

10.1 Hitos del proyecto

ID	Nombre del hito	Semana	Criterio de completado
M0	Gramática congelada	S1	El archivo grammar.ebnf está completo y los 35 keywords están definidos sin cambios pendientes
M1	Hola mundo ejecutable	S4	اطبع("مرحبا بالعالم") se ejecuta sin errores y produce el output correcto
M2	Fibonacci recursivo	S5	فيبوناتشي(10) devuelve 55 en menos de 2 segundos
M3	Clases y herencia	S5	El ejemplo حساب_بنكي del CU-04 se ejecuta correctamente con encadenamiento de métodos
M4	Errores en árabe	S7	Al menos 25 tipos de error distintos, todos con mensaje en árabe con número de línea y sugerencia
M5	Suite 50 tests	S8	Los 50 programas de test se ejecutan sin errores. Cobertura de tests del compilador > 80%
M6	CLI funcional	S8	لغة_العرب main.عرب ejecuta el programa desde terminal. لغة_العرب --repl abre modo interactivo
M7	Extensión VSCode	S9	Syntax highlighting funciona para todos los keywords en archivos .عرب abiertos en VSCode
M8	LSP diagnósticos	S10	Los errores del compilador aparecen subrayados en rojo en VSCode en tiempo real mientras se escribe
M9	Documentación completa	S11	La referencia del lenguaje en árabe cubre el 100% de los keywords, tipos y stdlib con ejemplos
M10	Release pública	S12	v0.1.0 publicada en GitHub con extensión VSCode en el Marketplace. Cualquier persona puede instalar y usar el lenguaje

10.2 Métricas de éxito

Métrica	Objetivo	Mínimo aceptable	Medición
Tests de integración pasando	50 / 50 (100%)	40 / 50 (80%)	pytest --tb=short
Cobertura del compilador	>= 85%	>= 70%	pytest --cov
Keywords implementados	35 / 35	28 / 35	tests/test_keywords.py
Tipos de error en árabe	25 mensajes distintos	15 mensajes	errors.py
Velocidad: fib(30)	< 3 segundos	< 10 segundos	time لغة_العرب fib.عرب
Velocidad arranque CLI	< 0.5 segundos	< 2 segundos	time لغة_العرب --version

Métrica	Objetivo	Mínimo aceptable	Medición
Extensión VSCode instalable	Sí, desde Marketplace	Sí, instalación manual	Manual test
Autocompletado de keywords	100% de keywords	80% de keywords	Manual test
Docs en árabe (cobertura)	100% keywords+stdlib	70%	Revisión manual
Programas de ejemplo	50 programas	30 programas	Recuento en /examples
Estrelllas GitHub (30 días)	> 100	> 20	GitHub API

11. Riesgos y Mitigaciones

Riesgo R01 — Complejidad del texto árabe bidireccional (bidi)

Nivel: ALTO. Probabilidad: ALTA. Impacto: ALTO.

El árabe usa escritura de derecha a izquierda (RTL), pero los números y operadores matemáticos (como 3.14 o >=) son de izquierda a derecha (LTR). Esto crea texto bidireccional (bidi) que puede confundir a los editores de texto, a las herramientas de regex y a los parsers.

Ejemplo del problema: el string "42 = متغير س" parece "42 = ريبغتم س" en un editor que interpreta mal el bidi. El lexer debe procesar la secuencia de codepoints en orden de memoria (que es RTL para el árabe), no en orden de visualización.

Mitigación R01

1. El Lexer trabaja siempre con índices de caracteres en la cadena Python, que son invariantes respecto a la dirección visual.
2. Antes de tokenizar, el texto se normaliza con unicodedata.normalize("NFC") para eliminar formas equivalentes.
3. Los números siempre son ASCII 0-9 — no hay dígitos árabes en el lenguaje, lo que elimina la ambigüedad más común.
4. El IDE configura Monaco Editor con direction: rtl explícito para el lenguaje lughat-al-arab.
5. Tests específicos de bidi: casos con mezcla de árabe + números + operadores en la misma línea.

Riesgo R02 — Inestabilidad del diseño del lenguaje

Nivel: ALTO. Probabilidad: MEDIA. Impacto: ALTO.

El error más común en proyectos de compiladores hechos por una sola persona es cambiar el diseño del lenguaje a mitad del proyecto. Añadir un nuevo keyword en la semana 6 obliga a modificar el lexer, el parser, el analizador semántico y el intérprete. Cada cambio puede introducir regresiones en los tests existentes.

Mitigación R02

1. REGLA DE ORO: la lista de keywords se congela en la semana 1. No se añade ningún keyword nuevo hasta v0.2.

2. Cualquier feature dudosa va a una lista "reservada" de keywords que el lexer reconoce pero no implementa.

3. La gramática EBNF se escribe antes de escribir una línea de código del compilador.

4. Los 10 programas de ejemplo se escriben a mano (sin ejecutar) en la semana 1 para validar que la sintaxis es cómoda.

5. Los cambios al AST en fases 2 y 3 solo pueden AÑADIR campos opcionales, nunca cambiar campos existentes.

Riesgo R03 — Tiempo limitado por obligaciones académicas

Nivel: ALTO. Probabilidad: ALTA. Impacto: MEDIO.

El proyecto se desarrolla compaginando con 1º de Bachillerato. Los períodos de exámenes (enero, mayo, junio) pueden reducir las horas disponibles a menos de 5 por semana, poniendo en riesgo el roadmap.

Mitigación R03

1. Semanas 1-4 son las más críticas — el lexer y parser son el 60% del trabajo. Priorizar estas semanas sobre cualquier otra actividad del proyecto.

2. En semanas de exámenes: pausar el IDE (semanas 9-12) y solo hacer tests del compilador (más cortos y predecibles).

3. Las semanas S8 y S12 son buffers explícitos en el roadmap. No asignarles trabajo nuevo.

4. El criterio mínimo aceptable es M5 (50 tests pasando en S8) — si se llega a ese hito, el proyecto es un éxito aunque el IDE quede incompleto.

5. El IDE en VSCode es más rápido de implementar que un IDE propio por esto — 3 días para lo básico, 2 semanas para LSP completo.

Riesgo R04 — Rendimiento del intérprete Python

Nivel: MEDIO. Probabilidad: MEDIA. Impacto: BAJO.

El intérprete de árbol en Python es intrínsecamente lento — ejecutar Lughat Al-Arab sobre Python es comparable a ejecutar Python sobre Python. Para programas grandes o algoritmos intensivos, esto puede resultar en tiempos de ejecución 50-100× más lentos que Python nativo.

Mitigación R04

1. El MVP es para programas educativos, no para producción. Fibonacci(30) en 3 segundos es aceptable para un estudiante.
2. Para el criterio de aceptación de velocidad se usa fib(30) < 3 segundos — alcanzable con el intérprete Python.
3. Si el rendimiento es un problema, se puede añadir memoización automática en el intérprete sin cambiar el lenguaje.
4. La versión 1.0 con Rust+LLVM resolverá el problema de rendimiento permanentemente.

Riesgo R05 — Soporte RTL incompleto en Monaco Editor

Nivel: MEDIO. Probabilidad: MEDIA. Impacto: MEDIO.

Monaco Editor (el núcleo de VSCode) no tiene soporte RTL nativo completo. Algunas operaciones como la selección de texto, el cursor y el movimiento de palabras pueden comportarse incorrectamente en texto árabe.

Mitigación R05

1. VSCode 1.85+ tiene mejoras significativas de soporte RTL. Usar siempre la versión más reciente.
2. La extensión configura explícitamente "editor.unicodeHighlight.ambiguousCharacters": false para evitar falsos positivos.
3. Si el soporte RTL es insuficiente en algún área específica, documentarlo como "known limitation" en v0.1 y planificar fix en v0.2.
4. Como fallback: el compilador funciona 100% desde la línea de comandos (CLI) independientemente del IDE.

12. Visión a Largo Plazo

12.1 Hoja de ruta post-MVP

Versión	Período	Enfoque principal	Features clave
v0.1	Mes 1-3	MVP — intérprete Python	Lexer+Parser+Intérprete, 50 tests, extensión VSCode básica
v0.2	Mes 4-5	Estabilización	Bugfixes, más ejemplos, LSP completo, tipado estático opcional
v0.3	Mes 6-8	Compilador Rust	Portar lexer+parser a Rust, AST como enum Rust
v0.4	Mes 9-11	LLVM backend	Generación de LLVM IR, compilación a binario nativo x86_64
v0.5	Mes 12-15	Concurrencia	Async/await árabe (انتظر / غير_متزامن), threads, canales
v0.6	Mes 16-18	IA y datos	مصفوفة (tensor), درب/تنبا como primitivos, bindings BLAS
v1.0	Mes 18-24	Ecosistema completo	Package manager, docs completas, comunidad, estabilidad

12.2 Compilador Rust + LLVM (v0.3-v0.4)

Una vez estabilizado el diseño del lenguaje con el intérprete Python, se porta el compilador a Rust. La arquitectura es la misma (Lexer → Parser → AST → Analizador semántico), pero el backend cambia de intérprete de árbol a generación de LLVM IR.

LLVM (Low Level Virtual Machine) es el compilador de compiladores de la industria. Lo usan Swift, Rust, Clang (C/C++), Kotlin Native, entre otros. Al generar código LLVM IR, Lughat Al-Arab obtiene acceso inmediato a décadas de optimizaciones: inlining, loop unrolling, vectorización SIMD, eliminación de código muerto, etc.

```
// Ejemplo: اطبع("مرحبا") se convierte en LLVM IR:

@str_مرحبا = global [13 x i8] c"\D9\85\D8\B1\D8\AD\D8\A8\D8\A7\0A\00"

define i32 @main() {
entry:
    %ptr = getelementptr [13 x i8], [13 x i8]* @str_مرحبا, i32 0, i32 0
    call i32 @puts(i8* %ptr)
    ret i32 0
}
```

```
declare i32 @puts(i8*)
```

12.3 Sistema de IA nativo (v0.6)

En la versión 0.6, Lughat Al-Arab añadirá soporte nativo para inteligencia artificial. Los keywords درب y تنبا dejarán de ser llamadas a librerías externas y se convertirán en primitivos del lenguaje con semántica bien definida.

El tipo مصفوفة (tensor/matriz multidimensional) se añadirá como tipo primitivo con operaciones integradas:

```
// مصفوفة: tensor nativo en Lughat Al-Arab v0.6
3 ,224 ,224)مصفوفة = متغير الصورة)  // tensor 224x224x3 (imagen RGB)

// operaciones tensoriales nativas
0.5 * الصورة = متغير نتيجة          // escalar
متغير transpuesto = الصورة.قلب()     // transposicion
متغير agrupado = الصورة.دمج(اخرى)    // concatenacion

// definicion de modelo con sintaxis nativa
نموذج رؤية_الصور {
    32)طبقة تحويلية, kernel: 3)   // Conv2D
    64)طبقة تحويلية, kernel: 3)
    طبقة مجمعة()                   // MaxPool
    128)طبقة كثيفة)
    سوفت_ماكس" :تفعيل ,10)طبقة كثيفة")
}

// entrenamiento con sintaxis declarativa
درب رؤية_الصور على بيانات_التدريب {
    حقب: 50
    معدل_التعلم: 0.001
    دالة_الخسارة: "كروس_انتروبي"
    محسن: "ادم"
}
```

12.4 Package manager: مدير الحزم

El ecosistema de Lughat Al-Arab en v1.0 incluirá un package manager propio llamado مدير الحزم (mudeer al-huzam). Similar a cargo de Rust o pip de Python, permite instalar, publicar y gestionar dependencias de proyectos:

```
// حزمة.عرب — equivalente a package.json o Cargo.toml
```

```
الحزمة {
    الاسم: "محلل_النصوص"
    الاصدار: "1.2.0"
    الوصف: "مكتبة لتحليل النصوص العربية"
    المؤلف: "احمد الكريمي"
    الاعتمادات {
        "رياضيات_متقدمة": "2.0^",
        "معالجة_اللغة": "1.5^"
    }
}

// Comandos del package manager (en terminal)
// مدير انشا مشروعي          -- nuevo proyecto
// مدير اضف رياضيات_متقدمة  -- instalar dependencia
// مدير بناء                 -- compilar proyecto
// مدير اختبر                -- ejecutar tests
// مدير نشر                  -- publicar en el registro
```

13. Próximos Pasos Inmediatos

Esta sección define exactamente qué hacer esta semana, antes de escribir una sola línea de código del compilador. El orden es importante — no saltar pasos.

Paso 1 — Crear el repositorio (Día 1, 30 minutos)

```
mkdir lughat-al-arab
cd lughat-al-arab
git init
mkdir compiler ide examples docs tests
touch README.md compiler/lexer.py compiler/parser.py
touch compiler/ast_nodes.py compiler/interpreter.py
touch compiler/semantic.py compiler/errors.py
touch tests/test_lexer.py tests/test_parser.py tests/test_integration.py
git add . && git commit -m "estructura inicial del proyecto"
gh repo create lughat-al-arab --public --push
```

Paso 2 — Escribir la gramática (Día 1-2, 6 horas)

Copiar la gramática EBNF de la sección 5.4 de este PRD en un archivo docs/grammar.ebnf. Leerla línea por línea y verificar que cada construcción del lenguaje que quieres tiene su regla. Añadir cualquier construcción que falte. NO implementar nada aún.

Paso 3 — Escribir 10 programas de ejemplo (Día 2-3, 4 horas)

Antes de escribir el lexer, escribir a mano 10 programas en Lughat Al-Arab. Guardarlos en examples/ con nombres descriptivos. Esto valida que la sintaxis se siente natural al escribirla. Si algo resulta incómodo de escribir, cambiar la sintaxis AHORA, no después de implementar el compilador.

- examples/01_hola_mundo.مرحبا("اطبع — عرب")
- examples/02_variables.ثابت ,متغير — عرب, tipos
- examples/03_condicionales.اذا/واذا/والا — عرب anidados
- examples/04_bucles.كرر ,بينما ,لكل — عرب
- examples/05_funciones.ارجع d— عرب ,الة, recursión
- examples/06_clases.يرث ,جديد ,هذا ,صنف — عرب
- examples/07_listas.مجموعة ,قاموس ,قائمة — عرب
- examples/08_errores.ارمي ,التقط ,حاول — عرب

- examples/09_modulos.عرب — استورد, رياضيات, نصوص
- examples/10_fibonacci.عرب — algoritmo recursivo completo

Paso 4 — Implementar el Lexer (Día 3-7, 10 horas)

Con la gramática y los ejemplos listos, implementar compiler/lexer.py siguiendo exactamente el código de la sección 6.2 de este PRD. El Lexer debe tokenizar correctamente los 10 programas de ejemplo.

Puntos críticos del Lexer a verificar

• Normalización NFC con unicodedata.normalize("NFC", codigo) — primera línea del constructor.

• El rango Unicode árabe: 0x0600-0x06FF. Sin esto, los identificadores árabes no se reconocen.

• Hamza equivalente: "اذا" y "إذا" deben producir el MISMO tipo de token (KW_AZDA).

• Los números son siempre ASCII 0-9, nunca indoarábigos. Si alguien escribe ٤٢, el lexer da error E009.

• El lexer debe trackear número de línea y columna para todos los tokens — es crítico para los errores.

Paso 5 — Tests del Lexer (Día 7-8, 5 horas)

Antes de pasar al parser, escribir al menos 50 tests para el lexer en tests/test_lexer.py usando pytest. Los tests deben cubrir: todos los keywords, identificadores árabes con y sin hamza, números enteros y flotantes, strings con caracteres árabes, comentarios, todos los operadores, casos límite (archivo vacío, línea con solo un comentario, identificador de un solo carácter).

Criterio de "listo para continuar"

El Lexer está listo para pasar al Parser cuando:

1. Los 50 tests de test_lexer.py pasan al 100%.
2. Los 10 programas de ejemplo se tokenizan sin errores.
3. Los mensajes de error del lexer (E009, E010) aparecen con número de línea y columna correctos.
4. La cobertura de test del lexer es > 90% (pytest --cov compiler/lexer.py).

14. Apéndice — Referencia Completa del Lenguaje

A. Tabla de operadores con ejemplos

Operador	Nombre árabe	Tipo	Ejemplo	Resultado
+	جمع	Aritmético	5 + 3	8
+	ربط نص	Cadena	"احمد" + " " + "مرحبا"	"مرحبا احمد"
-	طرح	Aritmético	10 - 4	6
*	ضرب	Aritmético	3 * 7	21
/	قسمة	Aritmético	15 / 4	3.75
%	باقي القسمة	Aritmético	17 % 5	2
**	اسس / قوة	Aritmético	2 ** 8	256
==	يساوي	Comparación	"احمد" == "احمد"	صح
!=	لا يساوي	Comparación	5 != 3	صح
<	اصغر من	Comparación	3 < 10	صح
<=	اصغر او يساوي	Comparación	5 <= 5	صح
>	اكبر من	Comparación	8 > 3	صح
>=	اكبر او يساوي	Comparación	7 >= 7	صح
و	منطقي AND	Lógico	صح و خطا	خطا
او	منطقي OR	Lógico	صح او خطا	صح
ليس	منطقي NOT	Lógico (unario)	ليس صح	خطا
=	تخصيص	Asignación	متغير س = 5	—
+=	جمع وتخصيص	Asignación compuesta	س += 3	س = س + 3
-=	طرح وتخصيص	Asignación compuesta	س -= 2	س = س - 2
*=	ضرب وتخصيص	Asignación	س *= 4	س = س * 4

Operador	Nombre árabe	Tipo	Ejemplo	Resultado
		compuesta		
/=	قسمة وتخصيص	Asignación compuesta	س =/ 2	س = س / 2

B. Referencia completa de la librería estándar

رياضيات — Módulo matemático

Función	Parámetros	Retorna	Ejemplo
جذر(س)	عدد >= 0	عدد	جذر(16) → 4.0
اسس(قاعدة, اس)	عددان	عدد	اسس(2, 10) → 1024
لوغاريتم(س)	عدد > 0	عدد	لوغاريتم(2.718) → 1.0
لوغاريتم(س, قاعدة)	عددان > 0	عدد	لوغاريتم(100, 10) → 2.0
جيب(زاوية)	عدد (radianes)	عدد	جيب(pi / 2) → 1.0
جيب_تمام(زاوية)	عدد (radianes)	عدد	جيب_تمام(0) → 1.0
قيمة_مطلقة(س)	عدد	عدد >= 0	قيمة_مطلقة(-7) → 7
دور(س, عدد_الخانات)	عدد, عدد_صحيح	عدد	دور(3.14159, 2) → 3.14
اقصى(قائمة)	قائمة من الارقام	عدد	اقصى([3,1,4,1,5]) → 5
ادنى(قائمة)	قائمة من الارقام	عدد	ادنى([3,1,4,1,5]) → 1
مجموع(قائمة)	قائمة من الارقام	عدد	مجموع([1,2,3,4]) → 10
pi	—	عدد	3.14159265358979...
e	—	عدد	2.71828182845904...

نصوص — Módulo de cadenas de texto

Función	Parámetros	Retorna	Ejemplo
طول(نص)	نص	عدد_صحيح	طول("مرحبا") → 5
قطع(نص, بداية, نهاية)	نص, عدد, عدد	نص	قطع("مرحبا", 0, 3) → "مرح"
دمج(قائمة, فاصل)	قائمة من النصوص, نص	نص	دمج(["أ","ب"], "-") → "أ-ب"
ابحث(نص, كلمة)	نصان	عدد أو فارغ	ابحث("مرحبا", "حب") → 2
استبدل(نص, قديم, جديد)	ثلاثة نصوص	نص	استبدل("مرحبا", "مرح", "أهلا")

Función	Parámetros	Retorna	Ejemplo
مقسم(نص, فاصل)	نصان	قائمة	مقسم("a,b,c", ",") → ["a","b","c"]
يبدأ_بـ(نص, بادئة)	نصان	صح/خطا	يبدأ_بـ("مرحبا", "مرح") → صح
ينتهي_بـ(نص, لاحقة)	نصان	صح/خطا	ينتهي_بـ("سلام", "ام") → صح
حروف_كبيرة(نص)	نص	نص	حروف_كبيرة("hello") → "HELLO"
حروف_صغيرة(نص)	نص	نص	حروف_صغيرة("HELLO") → "hello"
قص_المسافات(نص)	نص	نص	قص_المسافات(" مرحبا ") → "مرحبا"
تحويل_نص(قيمة)	أي نوع	نص	تحويل_نص(42) → "42"
تحويل_عدد(نص)	نص	عدد	تحويل_عدد("42") → 42

C. Guía de migración desde Python

Para programadores que ya conocen Python y quieren aprender Lughat Al-Arab:

Python	Lughat Al-Arab	Nota
if condicion:	اذا (condicion) { }	Paréntesis obligatorios, llaves en lugar de :
elif condicion:	واذا (condicion) { }	—
else:	والا { }	—
while condicion:	بينما (condicion) { }	—
for x in lista:	لكل س في القائمة { }	—
def funcion(a, b):	دالة funcion(a, b) { }	—
return valor	ارجع valor	—
class MiClase:	صنف MiClase { }	—
self	هذا	Equivalente exacto
MiClase()	جديد MiClase()	Requiere كلمة جديد
class A(B):	صنف A يرث B { }	—
print("hola")	اطبع("مرحبا")	—
input("prompt")	اقرا("prompt")	—
True / False	صح / خطا	—
None	فارغ	—
try: ... except:	حاول { } التقط (e) { }	—
raise Exception()	ارمي(جديد خطا())	—
import math	استورد رياضيات	—
[1, 2, 3]	[1, 2, 3]	Idéntico
{"a": 1}	{"a": 1}	Idéntico

Python	Lughat Al-Arab	Nota
# comentario	// تعليق	// en lugar de #
a ** b	a ** b	Idéntico
a and b	a و b	—
a or b	a او b	—
not a	ليس a	—

D. Preguntas frecuentes (FAQ)

¿Puedo mezclar árabe e inglés en el mismo archivo?

Sí. Los identificadores pueden ser en inglés ASCII (x, counter, myFunction) y el código será válido. Sin embargo, el espíritu del lenguaje es escribir todo en árabe. El compilador no impone ninguna restricción sobre el idioma de los identificadores.

¿Funciona en Windows, macOS y Linux?

El compilador en Python funciona en cualquier sistema que tenga Python 3.11+. La extensión VSCode funciona en todos los sistemas que soporten VSCode (Windows, macOS, Linux). En la versión 1.0 con Rust, se distribuirán binarios precompilados para x86_64 y ARM64 en los tres sistemas operativos.

¿Cómo se escribe árabe en un teclado estándar?

La mayoría de sistemas operativos incluyen un teclado árabe estándar. En Windows: Configuración → Tiempo e idioma → Teclados. En macOS: Preferencias del Sistema → Teclado → Fuentes de entrada. En Ubuntu: Configuración → Región e idioma. VSCode detecta automáticamente el modo RTL cuando el sistema tiene un teclado árabe activo.

¿El lenguaje soporta Unicode completo en strings?

Sí. Los strings en Lughat Al-Arab son secuencias de codepoints Unicode (UTF-8 en disco). Pueden contener árabe, inglés, chino, emoji, o cualquier otro carácter Unicode válido. El compilador no restringe el contenido de los strings.

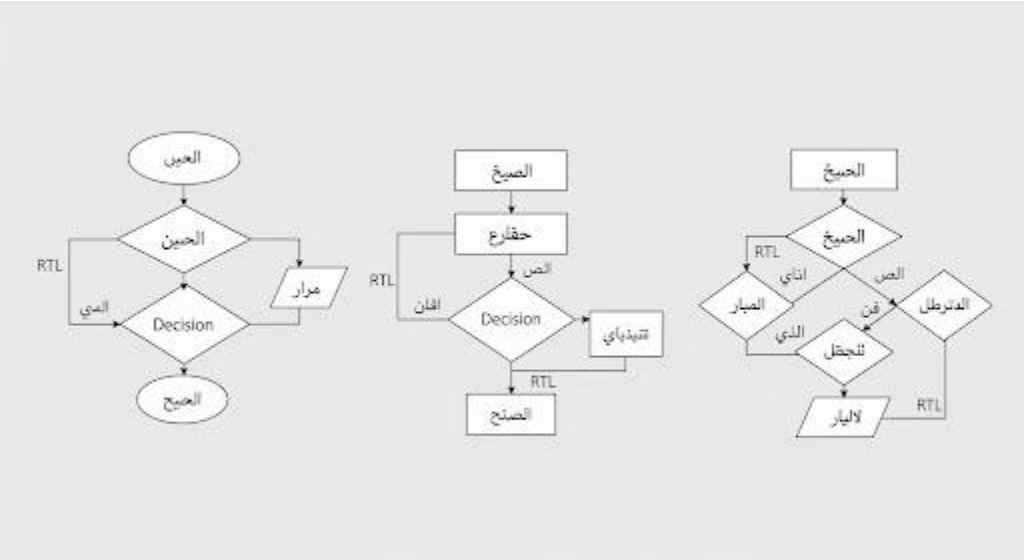
¿Cuándo estará disponible la versión compilada (nativa)?

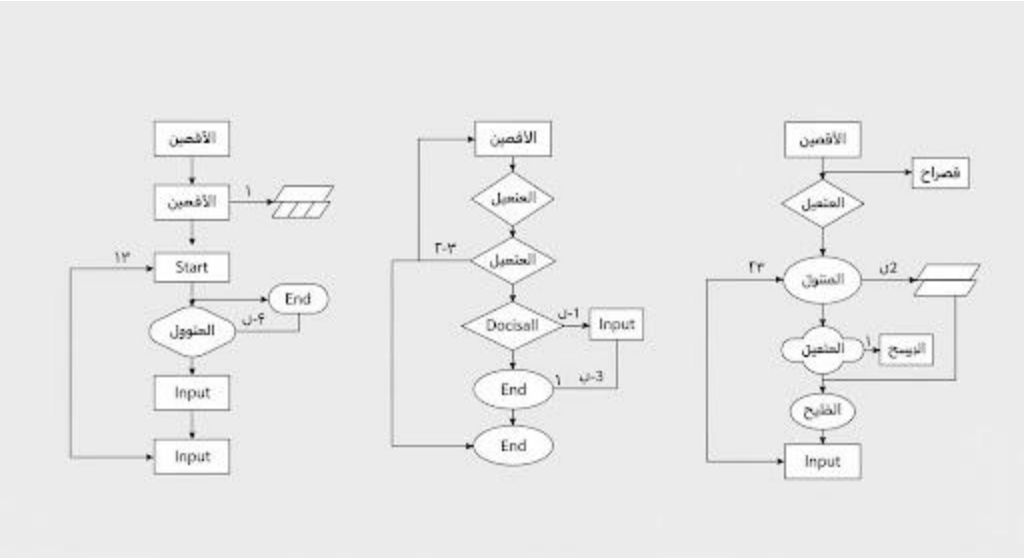
La versión compilada con Rust+LLVM está planificada para los meses 6-8 post-MVP (v0.3-v0.4). El MVP usa un intérprete de árbol en Python que es suficiente para aprendizaje y prototipado.

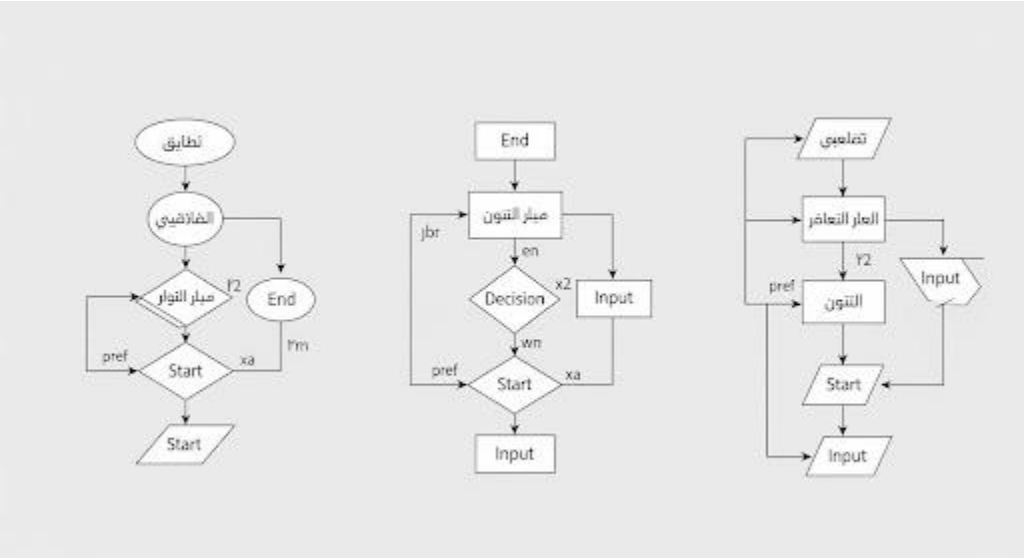
الفواج المحاول
المحان
الش لي
المحان
الفطلة المحليل
المجون
المحان
الخطوج
عي قرات
المحليل
في المحليل
RTL

الفواج المحليل
المحان
جي فرايع
الازة الحلين
الخفية
المحات
الفلش نقرق
الفواج المحليل
الشمات
المحليل
TL
IL

الفواج المحليل
الحنول
الدة
الحلدي
الحنية
فاية
الحنوي
المحول
RTL
IL


العين
الحين
RTL
مرار
المي
Decision
العيح
الصيخ
حقارع
RTL
الص
اقان
Decision
قتيذباي
RTL
الصنح
الحيخ
الحيخ
RTL
اناي
الص
المبار
قن
الدترطل
الذي
للجحل
لاليار
RTL


الأفصين
الأفصين
Start
End
المنوول
Input
Input
الأفصين
المتميل
المتميل
Docisall
Input
End
End
الأفصين
قصراح
المتميل
المنوول
المتميل
الربيح
الطابع
Input


تطابق
الفلاقيي
ميار التوار
End
Start
Start
End
ميار التنون
Decision
Input
Start
Input
تضامني
العار التعاضر
التنون
Input
Start
Input